HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN IOT VÀ ỨNG DỤNG

Giảng viên hướng dẫn:	**Nguyễn Quốc Uy**
Sinh viên thực hiện:	**Phạm Minh Thiện**
Mã sinh viên:	**B23DCAT285**
Nhóm lớp:	**16**

Hà Nội - 2026

LỜI CẢM ƠN

Để hoàn thành bài báo cáo bài tập lớn môn học “IoT và Ứng dụng”, em xin gửi lời cảm ơn chân thành tới thầy Nguyễn Quốc Uy, người đã trực tiếp giảng dạy và hướng dẫn em trong suốt quá trình học tập và thực hiện đề tài.

Những kiến thức thầy truyền đạt về hệ thống IoT, giao thức MQTT, thiết kế cơ sở dữ liệu, xây dựng API và phát triển giao diện Web là nền tảng để em xây dựng được hệ thống giám sát môi trường và điều khiển thiết bị được trình bày trong báo cáo này.

Do thời gian và kiến thức còn hạn chế, bài báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến góp ý của thầy để em hoàn thiện hơn.

Em xin chân thành cảm ơn!

Hà Nội, ngày 16 tháng 10 năm 2026

Sinh viên thực hiện

Phạm Minh Thiện

MỤC LỤC

LỜI CẢM ƠN **2**
CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN **4**
1.1. Đặt vấn đề: 4
1.2. Mục tiêu của đề tài: 4
1.3. Sơ đồ tổng quát của hệ thống: 5
CHƯƠNG 2: CÁC CHỨC NĂNG **6**
2.1. Các chức năng cơ bản: 6
2.2. Các thiết bị được sử dụng: 6
CHƯƠNG 3: GIAO DIỆN **7**
3.1. Đăng nhập: 7
3.2. Dashboard: 8
3.3. Data Sensor: 10
3.4. Action History: 12
3.5. Profile: 13
CHƯƠNG 4: THIẾT KẾ HỆ THỐNG **15**
4.1. Phần cứng: 15
4.1.1. Các thiết bị và thành phần: 15
4.1.2. Thiết kế mạch: 15
4.1.3. Lập trình cho mạch điều khiển ESP8266: 16
4.2. Backend: 21
4.2.1. Kiến trúc Backend: 21
4.2.2. Thiết kế cơ sở dữ liệu: 30
4.2.3. MQTT topic sử dụng: 34
4.3. Xây dựng API Docs: 35
4.3.1. Đăng nhập: 35
4.3.2. Lấy dữ liệu cảm biến mới nhất: 36
4.3.3. Lấy dữ liệu biểu đồ: 37
4.3.4. Lấy lịch sử dữ liệu cảm biến: 38
4.3.5. Điều khiển bật/tắt thiết bị: 39
4.3.6. Lấy lịch sử hoạt động: 40
4.4. Biểu đồ tuần tự: 42
4.4.1. Đăng nhập: 42
4.4.2. Xem dữ liệu cảm biến thời gian thực: 43
4.4.3. Xem biểu đồ dữ liệu cảm biến: 44
4.4.4. Điều khiển bật/tắt thiết bị: 45
4.4.5. Xem lịch sử dữ liệu cảm biến: 46
4.4.6. Xem lịch sử hoạt động: 47
4.5. Front-end: 47
4.6. Lấy dữ liệu từ server: 57
CHƯƠNG 5: KẾT LUẬN **58**
5.1. Các kết quả đã đạt được: 58
5.2. Hướng phát triển trong tương lai: 58

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN

1.1. Đặt vấn đề:

Trong những năm gần đây, sự phát triển của công nghệ thông tin và viễn thông đã đưa Internet vạn vật (IoT - Internet of Things) trở thành một trong những công nghệ nền tảng của cuộc Cách mạng công nghiệp lần thứ tư. IoT cho phép các thiết bị vật lý như cảm biến, vi điều khiển, thiết bị chấp hành kết nối với nhau qua mạng Internet để thu thập, trao đổi dữ liệu và phối hợp hoạt động mà không cần con người can thiệp trực tiếp.

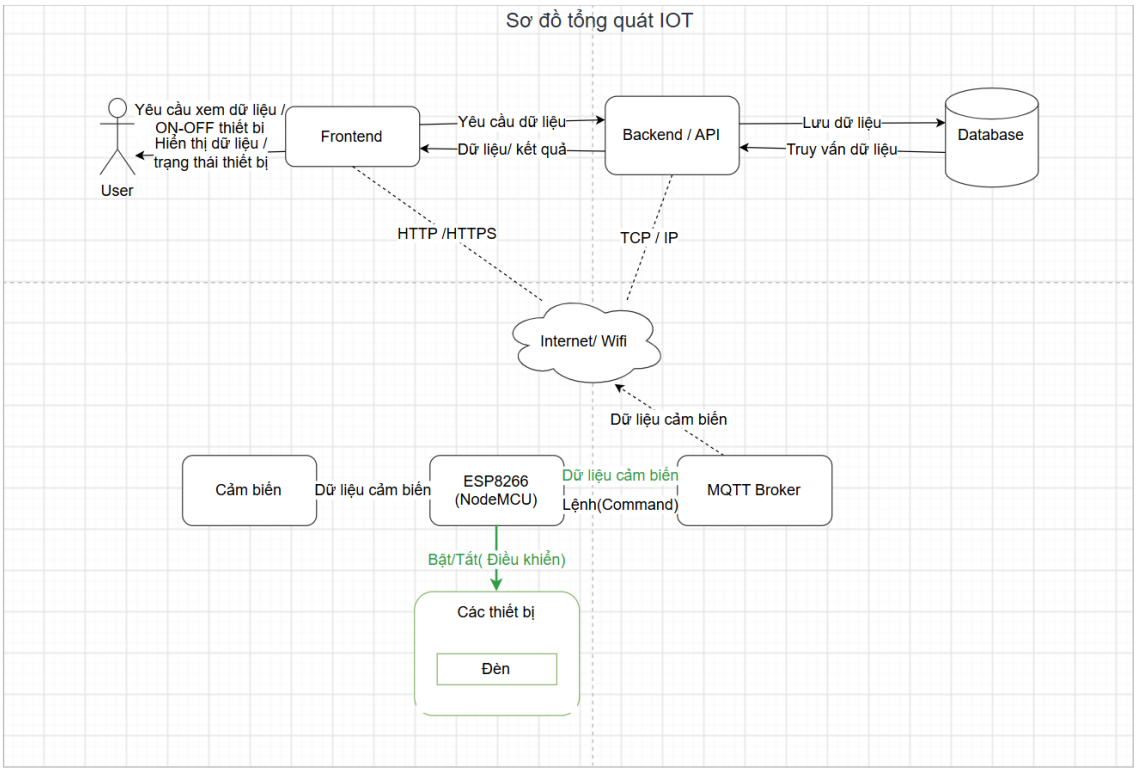
Trong đời sống hằng ngày, các thông số môi trường như nhiệt độ, độ ẩm và ánh sáng thay đổi liên tục, nhưng người dùng không thể quan sát trực tiếp tại mọi thời điểm. Việc bật/tắt thiết bị điện cũng thường phải thực hiện tại chỗ và không lưu lại được ai đã điều khiển, vào lúc nào, kết quả ra sao. Từ thực tế đó, đề tài xây dựng hệ thống **IoT_ThienPM**: một hệ thống giám sát môi trường và điều khiển đèn từ xa thông qua giao diện Web.

Hệ thống sử dụng vi điều khiển ESP8266 đọc dữ liệu từ cảm biến nhiệt độ, độ ẩm DHT11 và cảm biến ánh sáng (quang trở), gửi dữ liệu qua giao thức MQTT tới Backend Node.js để lưu vào cơ sở dữ liệu MySQL. Người dùng truy cập giao diện Web để xem số liệu, biểu đồ, điều khiển đèn và tra cứu lịch sử.

1.2. Mục tiêu của đề tài:

- Thu thập nhiệt độ, độ ẩm và cường độ ánh sáng theo chu kỳ từ các cảm biến gắn với ESP8266.
- Truyền dữ liệu cảm biến và lệnh điều khiển qua giao thức MQTT thông qua MQTT Broker Mosquitto.
- Xây dựng Backend cung cấp REST API có xác thực, lưu trữ dữ liệu cảm biến và lịch sử điều khiển vào MySQL.
- Xây dựng giao diện Web trực quan: hiển thị số liệu, biểu đồ, điều khiển bật/tắt đèn, tra cứu lịch sử có tìm kiếm, lọc và phân trang.

1.3. Sơ đồ tổng quát của hệ thống:



Hình 1.1. Sơ đồ tổng quát của hệ thống IoT

Người dùng làm việc trên giao diện Web (Frontend) bằng trình duyệt. Frontend trao đổi với Backend qua REST API (HTTP, dữ liệu JSON) và không truy cập trực tiếp cơ sở dữ liệu hay MQTT Broker. Backend chịu trách nhiệm xác thực, kiểm tra dữ liệu, xử lý nghiệp vụ, truy vấn MySQL và quản lý kết nối MQTT. ESP8266 gửi dữ liệu cảm biến lên Broker và nhận lệnh điều khiển từ Broker; nhờ đó các thành phần được tách rời và có thể thay thế độc lập.

CHƯƠNG 2: CÁC CHỨC NĂNG

2.1. Các chức năng cơ bản:

- **Đăng nhập:** người dùng đăng nhập bằng tài khoản trước khi sử dụng hệ thống; đăng xuất tại trang Profile.
- **Giám sát môi trường:** xem nhiệt độ, độ ẩm, ánh sáng mới nhất qua các thẻ số liệu và biểu đồ đường thể hiện sự biến thiên theo thời gian, mỗi đại lượng có trục giá trị riêng.
- **Điều khiển thiết bị:** bật/tắt đèn bằng hai nút ON/OFF; trạng thái hiển thị bằng chữ "Đang bật/Đang tắt", nút ON màu xanh khi đèn bật, OFF màu đỏ khi đèn tắt. Đèn (LIGHT_01) tương ứng LED thật nối với chân D0 của ESP8266.
- **Xem lịch sử dữ liệu cảm biến (Data Sensor):** lọc theo loại cảm biến, tìm kiếm chính xác theo giá trị hoặc thời gian, phân trang 5/10/20 bản ghi.
- **Xem lịch sử hoạt động (Action History):** lọc theo thiết bị, hành động (ON/OFF), trạng thái (Thành công/Thất bại/Đang chờ) và thời gian; phân trang.
- **Profile:** hiển thị thông tin cá nhân và đường dẫn tới GitHub, Figma, API Docs (Postman) và báo cáo PDF.

2.2. Các thiết bị được sử dụng:

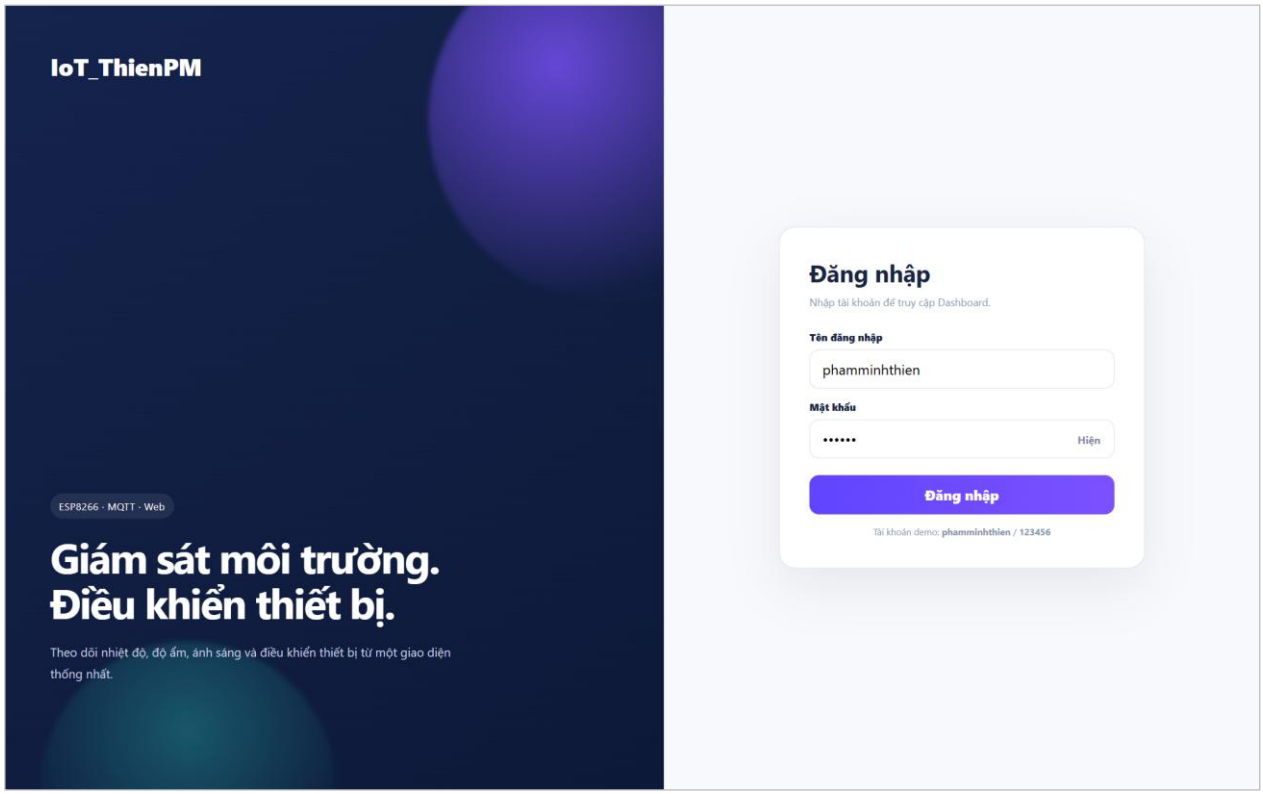
- Phần cứng:
  - ESP8266 NodeMCU 1.0 (ESP-12E)
  - Cảm biến đo nhiệt độ, độ ẩm DHT11
  - Cảm biến ánh sáng (module quang trở)
  - Đèn LED 5 mm và điện trở 220 Ω
  - Breadboard, dây nối, cáp micro USB
- Phần mềm:
  - Front-end: HTML, CSS, JavaScript, thư viện Chart.js
  - Back-end: Node.js, Express.js
  - Database: MySQL (chạy bằng Docker)
  - MQTT broker: Mosquitto
  - Công cụ: Arduino IDE, Visual Studio Code, Postman, Figma, Git/GitHub

CHƯƠNG 3: GIAO DIỆN

Giao diện Web gồm năm trang: Đăng nhập, Dashboard, Data Sensor, Action History và Profile, được thiết kế trên Figma trước khi lập trình. Các trang sau đăng nhập dùng chung thanh menu bên trái và thanh trên cùng có nút Hồ sơ, đồng hồ.

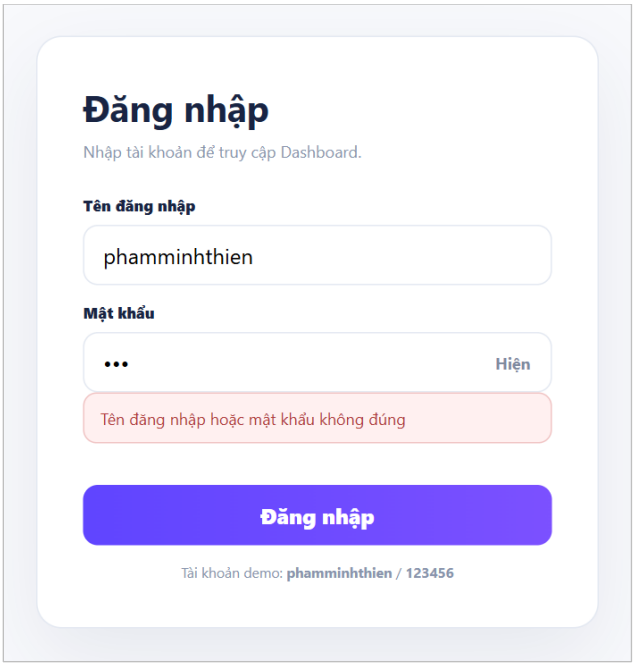
3.1. Đăng nhập:

Trang Đăng nhập gồm hai phần: bên trái giới thiệu hệ thống, bên phải là form nhập tên đăng nhập và mật khẩu, có nút Hiện/Ẩn mật khẩu. Đăng nhập thành công sẽ chuyển tới trang Dashboard; các trang khác đều yêu cầu đăng nhập trước khi truy cập.



Hình 3.1. Trang Đăng nhập

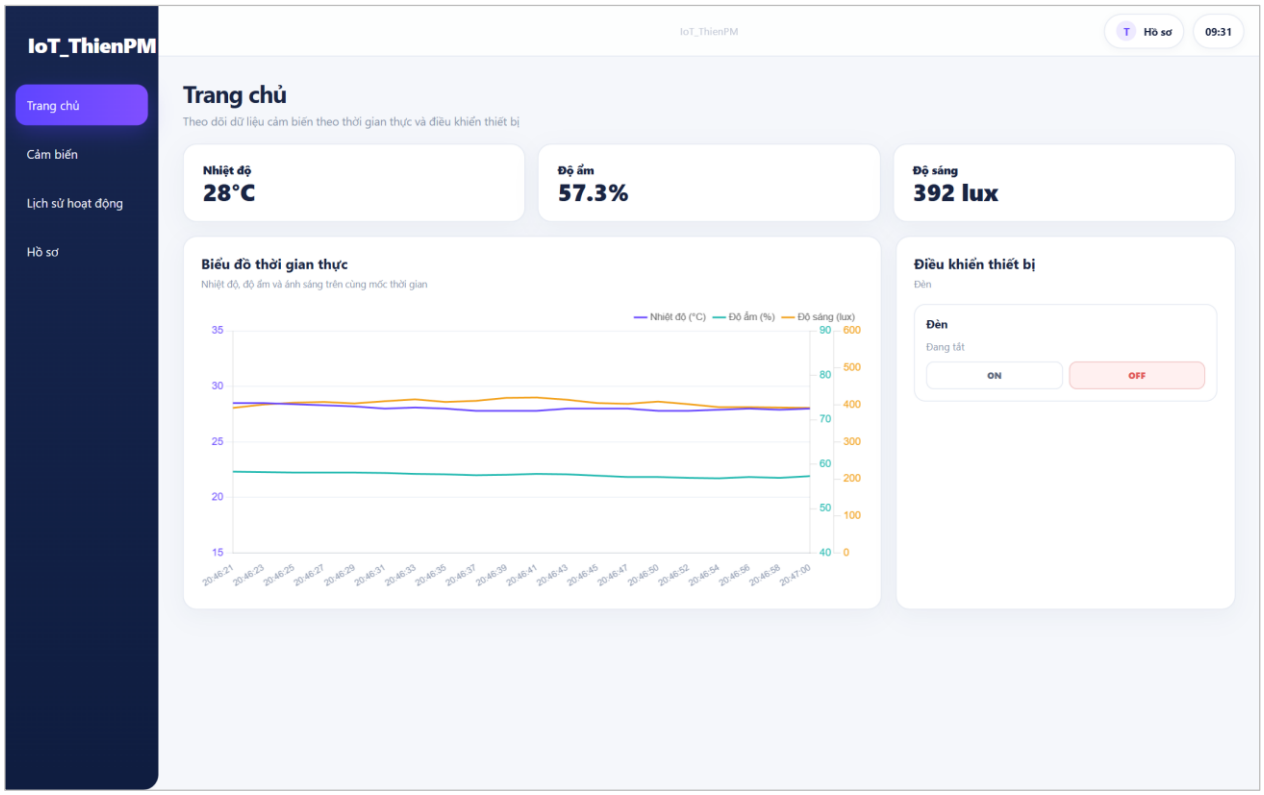
Khi nhập sai tài khoản hoặc mật khẩu, hệ thống hiển thị thông báo chung "Tên đăng nhập hoặc mật khẩu không đúng" và giữ người dùng ở trang đăng nhập.



Hình 3.2. Thông báo đăng nhập sai

3.2. Dashboard:

Trang Dashboard hiển thị trực quan các thông số thu thập từ cảm biến nhiệt độ, độ ẩm và ánh sáng. Dữ liệu được biểu diễn bằng thẻ số liệu và biểu đồ cho thấy sự thay đổi giá trị theo thời gian, đồng thời cung cấp các nút để người dùng điều khiển thiết bị.



Hình 3.3. Trang Dashboard

- Ba thẻ số liệu hiển thị giá trị mới nhất: Nhiệt độ (°C), Độ ẩm (%) và Độ sáng (lux).
- Biểu đồ đường thể hiện ba đại lượng trên cùng trục thời gian (tối đa 20 mốc gần nhất), mỗi đại lượng có một trục giá trị riêng cùng màu với đường:
    - Nhiệt độ: đường tím, trục trái, thang 15 – 35 °C.
    - Độ ẩm: đường xanh ngọc, trục phải, thang 40 – 90 %.
    - Độ sáng: đường cam, trục phải ngoài cùng, thang 0 – 600.
- Khi rê chuột vào một mốc thời gian, biểu đồ hiển thị chấm trên cả ba đường và một khung tooltip liệt kê đủ ba giá trị tại mốc đó.



Hình 3.4. Biểu đồ khi rê chuột vào mốc 20:46:29

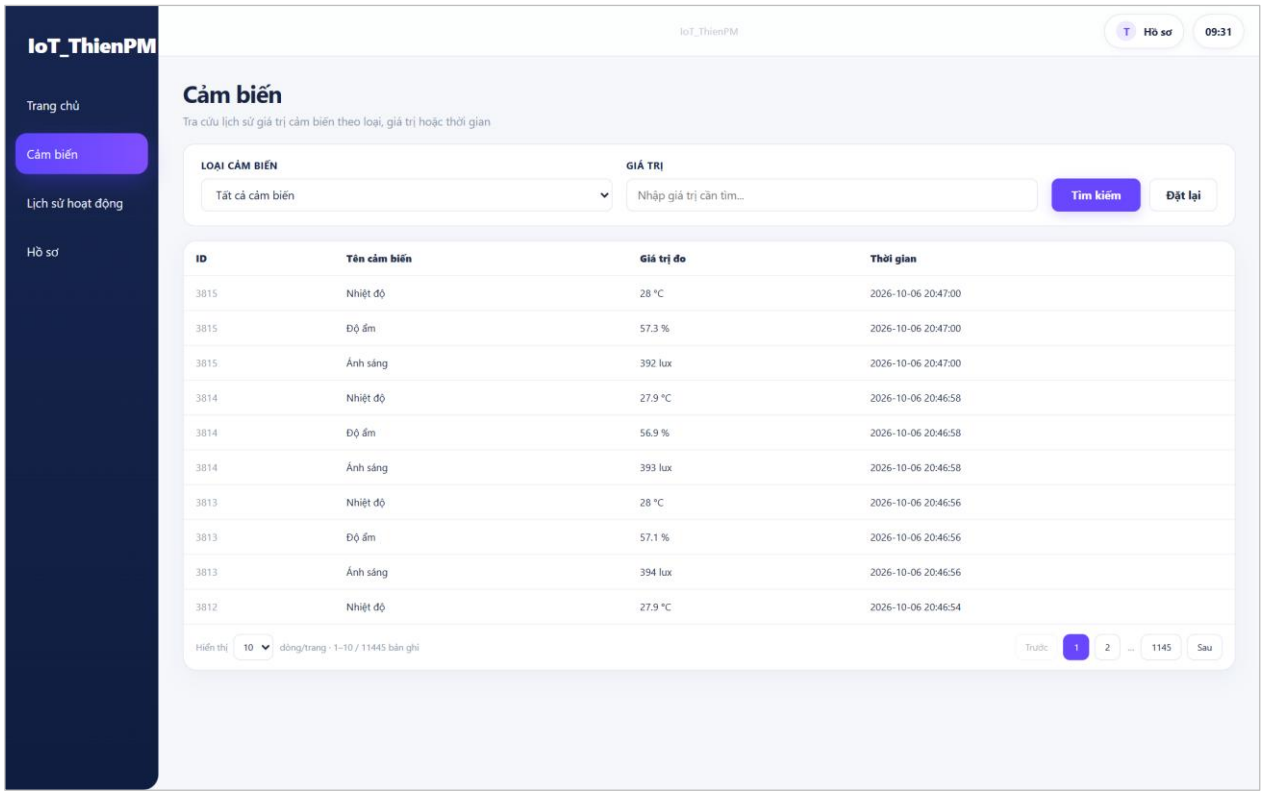
- Khu vực Điều khiển thiết bị hiển thị đèn kèm trạng thái "Đang bật" hoặc "Đang tắt" và hai nút ON/OFF; nút ứng với trạng thái hiện tại được tô màu: ON màu xanh khi đèn bật, OFF màu đỏ khi đèn tắt. Mỗi lần bấm, Frontend gửi lệnh tới Backend, trạng thái đèn được cập nhật và một dòng được ghi vào lịch sử hoạt động.



Hình 3.5. Điều khiển thiết bị trước (trái) và sau khi bật Đèn (phải)

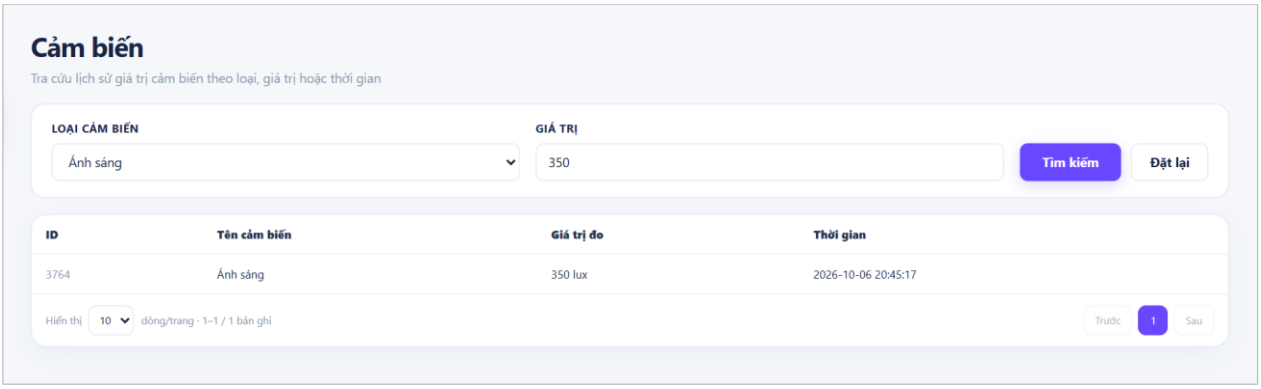
3.3. Data Sensor:

Trang Data Sensor (Cảm biến) lưu trữ lại các bản ghi dữ liệu thu được từ các cảm biến theo thời gian. Mỗi lần đo được tách thành ba dòng tương ứng ba cảm biến, bản ghi mới nhất được xếp lên đầu.



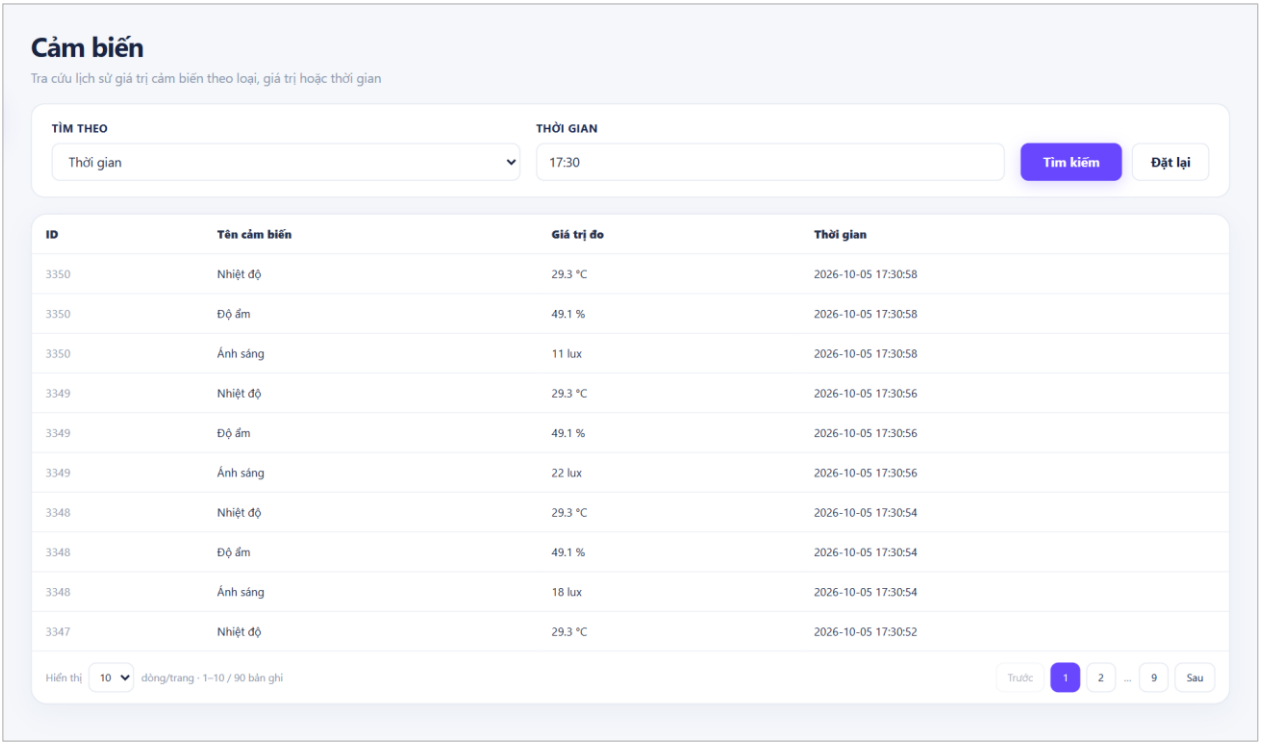
Hình 3.6. Trang Data Sensor

- Bảng dữ liệu cảm biến gồm 4 cột chính:
  - ID: số thứ tự bản ghi.
  - Tên cảm biến: Nhiệt độ, Độ ẩm hoặc Ánh sáng.
  - Giá trị đo: giá trị kèm đơn vị (°C, %, lux).
  - Thời gian: thời điểm ghi nhận dữ liệu, định dạng năm-tháng-ngày giờ:phút:giây (yyyy-MM-dd HH:mm:ss).
- Khung tìm kiếm gồm ô chọn loại cảm biến và ô nhập giá trị; hệ thống tìm các bản ghi có giá trị **bằng đúng** số đã nhập (ví dụ Ánh sáng = 350).



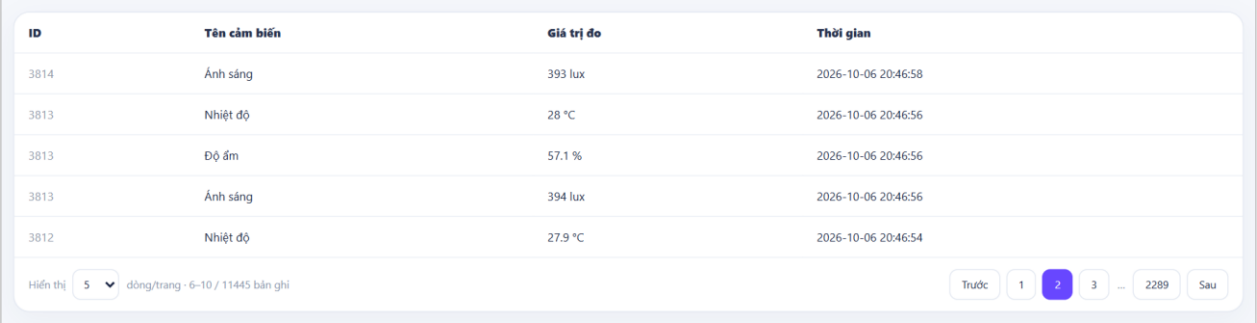
Hình 3.7. Tìm kiếm cảm biến Ánh sáng có giá trị 350

- Chọn mục "Thời gian" để tìm theo thời điểm đo. Các định dạng được hỗ trợ:
  - Giờ:phút[:giây], ví dụ 17:30 hoặc 17:30:15.
  - Khung giờ, ví dụ 17h.
  - Ngày theo dạng năm-tháng-ngày, ví dụ 2026-10-07; hoặc ngày kèm giờ, ví dụ 2026-10-07 17:30.



Hình 3.8. Tìm kiếm theo thời gian 17:30

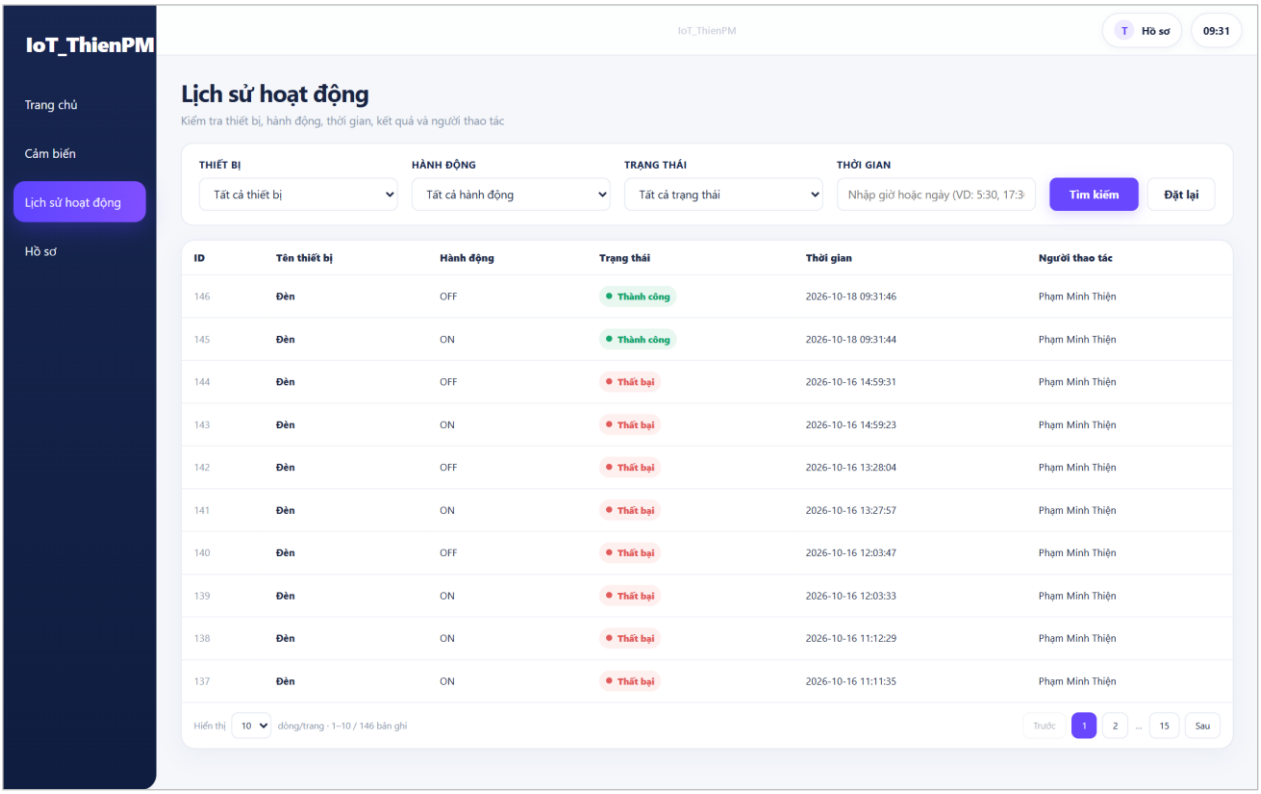
- Trang Data Sensor được phân trang, người dùng chọn 5, 10 hoặc 20 bản ghi mỗi trang; chân bảng hiển thị vị trí bản ghi đang xem và các nút Trước, số trang, Sau.



Hình 3.9. Phân trang 5 bản ghi mỗi trang

3.4. Action History:

Trang Action History (Lịch sử hoạt động) lưu trữ lại các bản ghi lịch sử bật/tắt thiết bị.

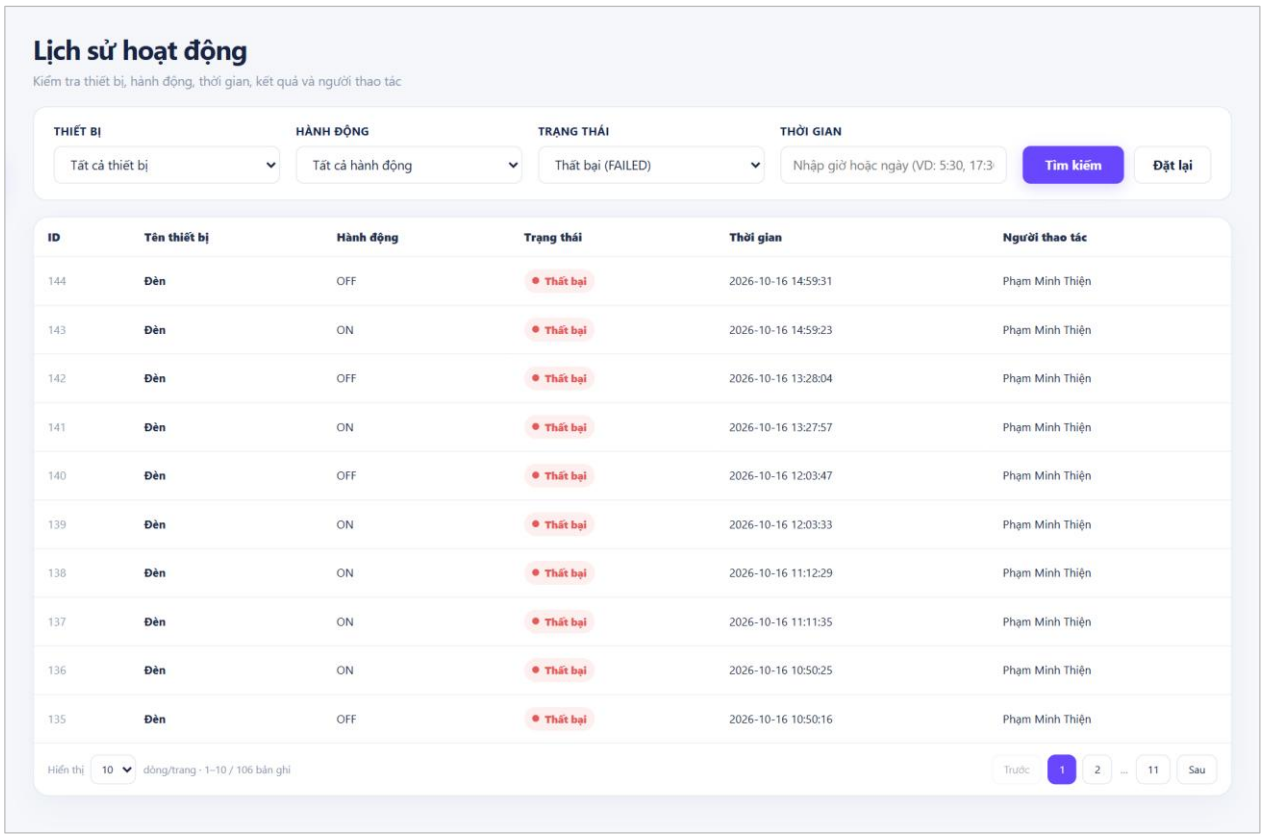


Hình 3.10. Trang Action History

- Bảng lịch sử hoạt động gồm 6 cột chính:
  - ID: số thứ tự bản ghi.
  - Tên thiết bị: đèn được điều khiển (Đèn).
  - Hành động: ON (bật) hoặc OFF (tắt).
  - Trạng thái: kết quả của lệnh, gồm Thành công (xanh), Thất bại (đỏ), Đang chờ (cam).
  - Thời gian: thời điểm thực hiện thao tác.

- Người thao tác: tên người dùng đã điều khiển.

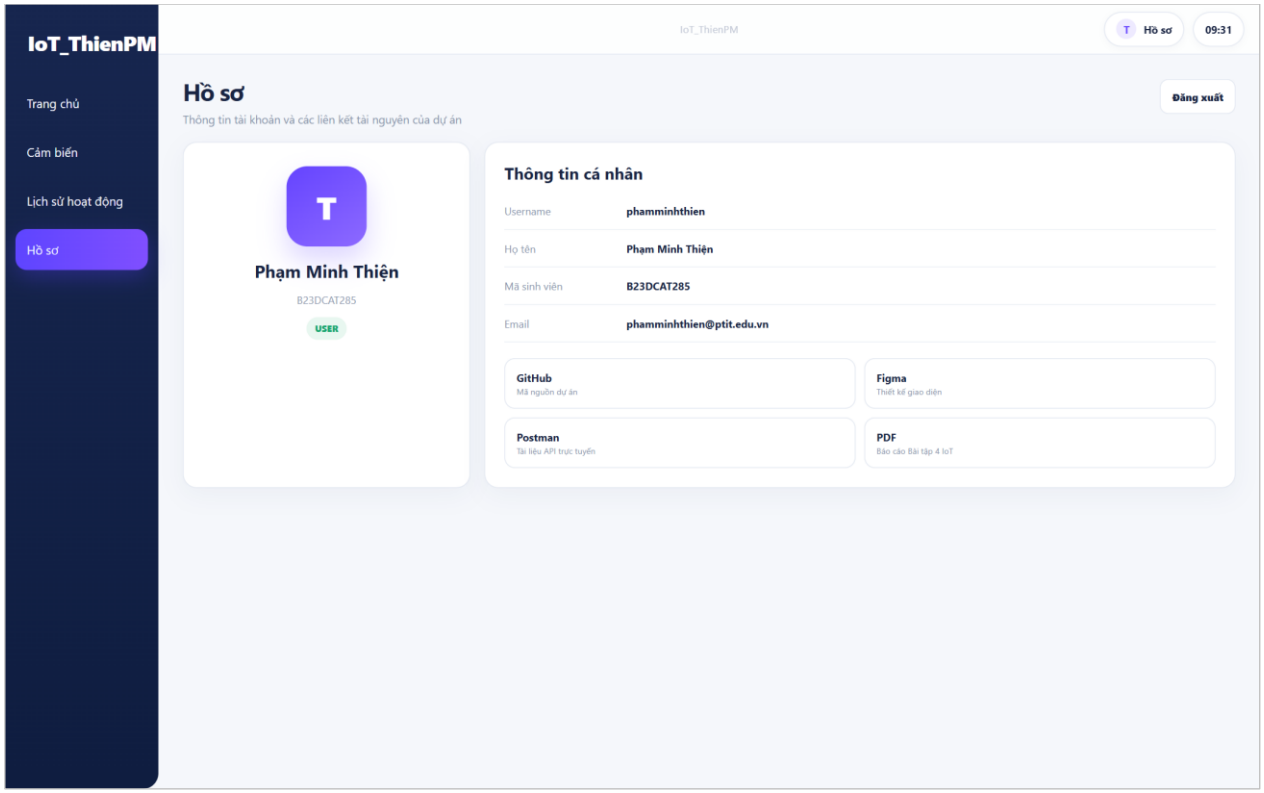
- Thanh lọc gồm bốn tiêu chí có thể kết hợp: Thiết bị, Hành động, Trạng thái và Thời gian. Bảng được phân trang giống trang Data Sensor.



Hình 3.11. Lọc các lần điều khiển có trạng thái Thất bại

3.5. Profile:

Trang Profile hiển thị các thông tin cá nhân (họ tên, mã sinh viên, username, email, vai trò) và các đường dẫn đến GitHub, Figma, API Docs (Postman) và báo cáo PDF. Nút Đăng xuất đưa người dùng về trang Đăng nhập.



Hình 3.12. Trang Profile

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG

4.1. Phần cứng:

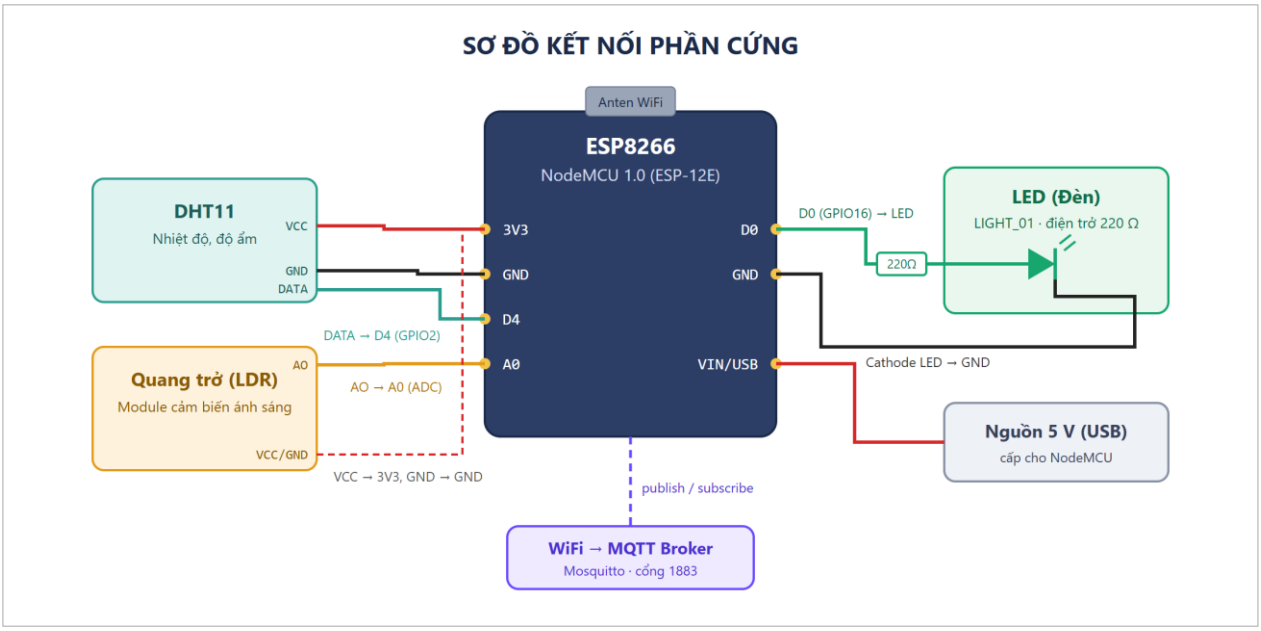
4.1.1. Các thiết bị và thành phần:

Mạch bao gồm các thành phần:

Bảng 4.1. Các thành phần phần cứng

Thành phần	Số lượng	Thông số chính	Vai trò
ESP8266 NodeMCU 1.0 (ESP-12E)	1	Wi-Fi 2.4 GHz, mức logic 3.3 V, 1 chân ADC (A0)	Vi điều khiển trung tâm: đọc cảm biến, kết nối Wi-Fi và MQTT, điều khiển LED
Cảm biến DHT11	1	Nhiệt độ 0 – 50 °C (±2 °C), độ ẩm 20 – 90 % (±5 %)	Đo nhiệt độ và độ ẩm
Module quang trở (LDR)	1	Ngõ ra tương tự, đọc qua ADC 10 bit (0 – 1023)	Đo cường độ ánh sáng
LED 5 mm + điện trở 220 Ω	1	Điều khiển bằng chân số	Đèn (thiết bị LIGHT_01)
Breadboard, dây nối, cáp micro USB	1 bộ	Nguồn 5 V từ cổng USB	Lắp mạch và cấp nguồn

4.1.2. Thiết kế mạch:



Hình 4.1. Thiết kế mạch

Bảng 4.2. Sơ đồ chân kết nối

Thiết bị	Chân thiết bị	Chân NodeMCU	Ghi chú
DHT11	VCC / GND / DATA	3V3 / GND / D4 (GPIO2)	Đọc số (digital)
Module quang trở	VCC / GND / AO	3V3 / GND / A0	Đọc tương tự (ADC)
LED (Đèn)	Anode (qua 220 Ω) / Cathode	D0 (GPIO16) / GND	Mức HIGH: bật đèn
Nguồn	USB 5 V	Cổng micro USB	Cấp nguồn cho cả mạch

4.1.3. Lập trình cho mạch điều khiển ESP8266:

Sử dụng Arduino IDE với các thư viện ESP8266WiFi, PubSubClient (MQTT) và DHT. Thông tin Wi-Fi và tài khoản MQTT được đặt riêng trong tệp secrets.h, không đưa lên GitHub. Chương trình gồm các phần chính sau:

- Khai báo thư viện, chân kết nối, mã cảm biến SENSOR_01, mã thiết bị LIGHT_01, các topic MQTT và chu kỳ gửi dữ liệu (2 giây) cũng như heartbeat (30 giây).

sketch_aug29a.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include "DHT.h"
#include <time.h>
  ...
// ---------- Cau hinh phan cung ----------
#define DHTPIN D4        // Data DHT11 noi D4 (GPIO2)
#define DHTTYPE DHT11    // Loai cam bien DHT11
#define LDRPIN A0        // Quang tro noi A0
#define LEDPIN D0        // LED noi D0 (GPIO16) - relay/den that duy nhat hien co

// ---------- Cau hinh WiFi va MQTT - CAN CHINH TRUOC KHI NAP ----------
// WIFI_SSID, WIFI_PASSWORD, MQTT_HOST, MQTT_PORT, MQTT_USER, MQTT_PASSWORD nam trong secrets.h
// (khong dua len git). Lan dau: chep secrets.example.h thanh secrets.h roi dien thong tin that.
#include "secrets.h"
const char* NODE_ID       = "esp8266-01";

// ---------- Ma cam bien / thiet bi khop voi bang sensors/devices trong CSDL ----------
// LIGHT_01 la thiet bi that duy nhat (LED tren chan D0).
const char* SENSOR_CODE   = "SENSOR_01";
const char* DEVICE_CODE   = "LIGHT_01";

// ---------- Topic MQTT theo dung thiet ke Chuong 2.8 cua bao cao ----------
String TOPIC_SENSOR_DATA;    // iot/sensors/{sensorCode}/data   - ESP8266 -> Backend
String TOPIC_DEVICE_COMMAND; // iot/devices/{deviceCode}/command - Backend -> ESP8266
String TOPIC_DEVICE_STATUS;  // iot/devices/{deviceCode}/status  - ESP8266 -> Backend (ACK)
String TOPIC_HEARTBEAT;      // iot/nodes/{nodeId}/heartbeat     - ESP8266 -> Backend

const unsigned long SENSOR_INTERVAL_MS    = 2000;  // Chu ky publish du lieu cam bien
const unsigned long HEARTBEAT_INTERVAL_MS = 30000; // Chu ky gui heartbeat
const unsigned long WIFI_RETRY_MS         = 10000; // Chu ky thu lai WiFi khi mat ket noi
const unsigned long MQTT_RETRY_MS         = 3000;  // Chu ky thu lai MQTT khi mat ket noi
```

Hình 4.2. Khai báo thư viện, chân kết nối và topic MQTT

- Hàm publishSensorData: đọc DHT11 và quang trở (lấy trung bình 10 lần đọc ADC để lọc nhiễu), bỏ qua lần đọc lỗi, đóng gói dữ liệu thành JSON kèm thời gian (đồng bộ qua NTP, múi giờ +07:00) rồi publish lên topic iot/sensors/SENSOR_01/data.

sketch_aug29a.ino

```
// Doc DHT11 + quang tro, publish JSON len topic telemetry theo dung dinh dang bao cao
void publishSensorData() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();
  int lightRaw = readLdrFiltered(10);

  if (isnan(humidity) || isnan(temperature)) {
    return;
  }

  String timestamp = getIsoTimestamp();
  if (timestamp.length() == 0) return; // Chua dong bo NTP xong thi bo qua chu ky nay

  String payload;
  payload.reserve(180);
  payload  = "{\"sensorCode\":\"" + String(SENSOR_CODE) + "\",";
  payload += "\"temperature\":" + String(temperature, 1) + ",";
  payload += "\"humidity\":" + String(humidity, 1) + ",";
  payload += "\"light\":" + String(lightRaw) + ",";
  payload += "\"timestamp\":\"" + timestamp + "\"}";

  mqttClient.publish(TOPIC_SENSOR_DATA.c_str(), payload.c_str(), false);

  Serial.print("Nhiet do: ");
  Serial.print(temperature, 1);
  Serial.print(" *C | Do am: ");
  Serial.print(humidity, 1);
  Serial.print(" % | Anh sang: ");
  Serial.print(lightRaw);
  Serial.print(" | Thoi gian: ");
  Serial.println(getDisplayTimestamp());
}
```

Hình 4.3. Đọc cảm biến và gửi dữ liệu lên MQTT

- Hàm mqttCallback và handleCommand: nhận lệnh JSON từ topic iot/devices/LIGHT_01/command, bật/tắt LED theo action ON/OFF, sau đó publishStatus gửi phản hồi (ACK) gồm actionCode, trạng thái thực tế và kết quả lên topic status để Backend đối chiếu.

sketch_aug29a.ino

```
// Publish ACK ve topic status voi cung actionCode nhan duoc (dung de Backend doi chieu UC04)
void publishStatus(const String& actionCode, bool success) {
  String payload;
  payload.reserve(180);
  payload  = "{\"actionCode\":\"" + actionCode + "\",";
  payload += "\"deviceCode\":\"" + String(DEVICE_CODE) + "\",";
  payload += "\"status\":\"" + String(ledOn ? "ON" : "OFF") + "\",";
  payload += "\"result\":\"" + String(success ? "SUCCESS" : "REJECTED") + "\",";
  payload += "\"confirmedAt\":\"" + getIsoTimestamp() + "\"}";
  mqttClient.publish(TOPIC_DEVICE_STATUS.c_str(), payload.c_str());
}

// Xu ly lenh ON/OFF, dung chung cho ca lenh tu Serial va tu MQTT
void handleCommand(String action, const String& actionCode, bool fromMqtt) {
  action.trim();
  action.toUpperCase();
  if (action.length() == 0) return;

  if (action == "ON") {
    applyLed(true);
    Serial.println("Bat den");
    if (fromMqtt) publishStatus(actionCode, true);
  } else if (action == "OFF") {
    applyLed(false);
    Serial.println("Tat den");
    if (fromMqtt) publishStatus(actionCode, true);
  } else {
    // AUTO hoac gia tri khac: node nay chua ho tro che do tu dong, tu choi lenh
    if (fromMqtt) publishStatus(actionCode, false);
  }
}
  ...
// Nhan lenh JSON tu topic command qua MQTT: {"actionCode":..,"deviceCode":..,"action":..,"mode":..}
void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String message;
  message.reserve(length);
  for (unsigned int i = 0; i < length; i++) {
    message += static_cast<char>(payload[i]);
  }
  if (String(topic) == TOPIC_DEVICE_COMMAND) {
    String actionCode = extractJsonString(message, "actionCode");
    String action = extractJsonString(message, "action");
    handleCommand(action, actionCode, true);
  }
}
```

Hình 4.4. Nhận lệnh điều khiển và gửi phản hồi trạng thái

- Hàm setup và loop: khởi tạo chân, cảm biến, Wi-Fi, NTP và MQTT; vòng lặp tự kết nối lại Wi-Fi/MQTT khi mất kết nối mà không dùng delay() chặn chương trình, gửi dữ liệu cảm biến và heartbeat theo chu kỳ.

sketch_aug29a.ino

```
224 void setup() {
225   Serial.begin(115200);
226   pinMode(LEDPIN, OUTPUT);
227   applyLed(false);
228
229   TOPIC_SENSOR_DATA    = "iot/sensors/" + String(SENSOR_CODE) + "/data";
230   TOPIC_DEVICE_COMMAND = "iot/devices/" + String(DEVICE_CODE) + "/command";
231   TOPIC_DEVICE_STATUS  = "iot/devices/" + String(DEVICE_CODE) + "/status";
232   TOPIC_HEARTBEAT      = "iot/nodes/" + String(NODE_ID) + "/heartbeat";
233
234   dht.begin();
235   startWiFi();
236   configTime(GMT_OFFSET_SEC, DAYLIGHT_OFFSET_SEC, "pool.ntp.org", "time.nist.gov");
237   mqttClient.setServer(MQTT_HOST, MQTT_PORT);
238   mqttClient.setCallback(mqttCallback);
239 }
240
241 void loop() {
242   maintainWiFi();
243   maintainMQTT();
244   if (mqttClient.connected()) {
245     mqttClient.loop(); // Bat buoc goi thuong xuyen de nhan lenh va giu keep-alive
246   }
247
248   handleSerialCommand();
249
250   unsigned long now = millis();
251   if (mqttClient.connected() && now - lastSensorPublish >= SENSOR_INTERVAL_MS) {
252     lastSensorPublish = now;
253     publishSensorData();
254   }
255   if (mqttClient.connected() && now - lastHeartbeat >= HEARTBEAT_INTERVAL_MS) {
256     lastHeartbeat = now;
257     publishHeartbeat();
258   }
259 }
```

Hình 4.5. Hàm setup và vòng lặp loop

Chương trình trên ESP8266 được cấu hình để thực hiện cơ chế publish/subscribe với Mosquitto Broker. Cách tiếp cận này thiết lập kênh giao tiếp hiệu quả giữa ESP8266 và Backend thông qua giao thức MQTT, một giao thức nhẹ, phù hợp với các thiết bị IoT có tài nguyên hạn chế.

4.2. Backend:

4.2.1. Kiến trúc Backend:

Backend được xây dựng bằng Node.js và Express.js, sử dụng các thư viện mysql2 (kết nối MySQL), mqtt (kết nối Broker), jsonwebtoken (tạo và kiểm tra JWT), bcryptjs (băm mật khẩu), dotenv và cors. Mã nguồn được tổ chức như sau:

Bảng 4.3. Tổ chức mã nguồn Backend

Tệp	Chức năng
`src/server.js`, `src/app.js`	Khởi động máy chủ HTTP cổng 3000, khai báo các nhóm API dưới tiền tố /api và xử lý lỗi chung.
`src/db.js`	Tạo pool kết nối MySQL.
`src/middleware/auth.js`	Kiểm tra JWT trong header Authorization cho các API cần đăng nhập.
`src/routes/auth.js`	Đăng nhập (so khớp mật khẩu bằng bcrypt, cấp JWT) và lấy thông tin người dùng.
`src/routes/sensors.js`	Dữ liệu cảm biến mới nhất, dữ liệu biểu đồ, danh sách dữ liệu có tìm kiếm, sắp xếp, phân trang.
`src/routes/devices.js`	Danh sách thiết bị và điều khiển bật/tắt.
`src/routes/history.js`	Lịch sử điều khiển có lọc, tìm kiếm và phân trang.
`src/mqttClient.js`	Subscribe dữ liệu cảm biến, phản hồi trạng thái, heartbeat; lưu dữ liệu vào MySQL; publish lệnh điều khiển.
`src/utils/*.js`	Hàm phân trang, ghép điều kiện SQL an toàn và tìm kiếm theo thời gian.

Một số đoạn mã nguồn chính của Backend:

src/server.js

```
import 'dotenv/config'
import { app } from './app.js'
import { pingDb } from './db.js'
import { connectMqtt } from './mqttClient.js'

async function main() {
  await pingDb()
  console.log('[db] connected')

  connectMqtt()

  const port = Number(process.env.PORT || 3000)
  app.listen(port, () => console.log(`[http] listening on http://localhost:${port}`))
}

main().catch((err) => {
  console.error('[startup] failed:', err)
  process.exit(1)
})
```

src/app.js

```
import express from 'express'
import cors from 'cors'
import authRoutes from './routes/auth.js'
import sensorRoutes from './routes/sensors.js'
import deviceRoutes from './routes/devices.js'
import historyRoutes from './routes/history.js'

export const app = express()

app.use(cors())
app.use(express.json())

app.get('/api/health', (req, res) => res.json({ status: 'ok' }))

app.use('/api/auth', authRoutes)
app.use('/api', sensorRoutes)
app.use('/api', deviceRoutes)
app.use('/api', historyRoutes)

app.use((req, res) => res.status(404).json({ message: 'Not found' }))

// eslint-disable-next-line no-unused-vars
app.use((err, req, res, next) => {
  // Body JSON sai cú pháp (express.json báo lỗi entity.parse.failed): lỗi của client, trả 400
  if (err.type === 'entity.parse.failed') {
    return res.status(400).json({ message: 'Dữ liệu JSON không hợp lệ' })
  }
  console.error('[error]', err)
  res.status(500).json({ message: 'Lỗi hệ thống' })
})
```

Hình 4.6. Khởi động máy chủ và khai báo các nhóm API (server.js, app.js)

src/db.js

```js
import mysql from 'mysql2/promise'
import 'dotenv/config'

export const pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  timezone: 'Z',
  dateStrings: false
})

export async function pingDb() {
  const conn = await pool.getConnection()
  try {
    await conn.query('SELECT 1')
  } finally {
    conn.release()
  }
}
```

src/middleware/auth.js

```js
import jwt from 'jsonwebtoken'

export function requireAuth(req, res, next) {
  const header = req.headers.authorization || ''
  const token = header.startsWith('Bearer ') ? header.slice(7) : null
  if (!token) return res.status(401).json({ message: 'Thiếu access token' })

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET)
    next()
  } catch {
    return res.status(401).json({ message: 'Access token không hợp lệ hoặc đã hết hạn' })
  }
}
```

src/utils/asyncRoute.js

```js
// Express 4 không tự bắt lỗi của handler async: lỗi (VD mất kết nối MySQL) thành
// unhandled rejection và làm tắt cả tiến trình Node. Bọc handler để chuyển lỗi
// sang middleware xử lý lỗi chung trong app.js (trả 500, server vẫn chạy).
export function asyncRoute(handler) {
  return (req, res, next) => Promise.resolve(handler(req, res, next)).catch(next)
}
```

Hình 4.7. Kết nối MySQL, kiểm tra JWT và bắt lỗi route (db.js, middleware/auth.js, utils/asyncRoute.js)

src/routes/auth.js

```js
router.post('/login', asyncRoute(async (req, res) => {
  const { username, password } = req.body || {}
  if (!username || !password) {
    return res.status(400).json({ message: 'Thiếu username hoặc password' })
  }

  const [[user]] = await pool.query(
    'SELECT user_id, username, password_hash, full_name, role FROM users WHERE username = ?',
    [String(username).trim()]
  )

  const genericError = { message: 'Tên đăng nhập hoặc mật khẩu không đúng' }
  if (!user) return res.status(401).json(genericError)

  const ok = await bcrypt.compare(password, user.password_hash)
  if (!ok) return res.status(401).json(genericError)

  const expiresIn = Number(process.env.JWT_EXPIRES_IN || 3600)
  const accessToken = jwt.sign({ userId: user.user_id, role: user.role }, process.env.JWT_SECRET, { expiresIn })

  res.json({
    accessToken,
    expiresIn,
    user: { userId: user.user_id, fullName: user.full_name, role: user.role }
  })
}))
```

Hình 4.8. API đăng nhập (routes/auth.js)

src/routes/sensors.js

```
// Lần đo mới nhất (thẻ Nhiệt độ / Độ ẩm / Độ sáng trên Dashboard)
router.get('/sensor-data/latest', requireAuth, asyncRoute(async (req, res) => {
  const { sensorCode } = req.query
  const [[row]] = await pool.query(
    `SELECT sd.data_id, s.sensor_code, s.sensor_name, sd.temperature, sd.humidity, sd.light, sd.status, sd.recorded_at
     ${SENSOR_DATA_FROM}
     ${sensorCode ? 'WHERE s.sensor_code = ?' : ''}
     ORDER BY sd.recorded_at DESC LIMIT 1`,
    sensorCode ? [sensorCode] : []
  )
  if (!row) return res.status(404).json({ data: null, message: 'Chưa có dữ liệu' })

  const dataAgeSeconds = Math.max(0, Math.round((Date.now() - new Date(row.recorded_at).getTime()) / 1000))
  res.json({ ...toMeasurementDto(row), stale: dataAgeSeconds > 30, dataAgeSeconds })
}))

// Biểu đồ Dashboard.
// - points > 0: chế độ thời gian thực, trả về N lần đo gần nhất (không gom nhóm)
// - còn lại: gom trung bình theo interval (1m | 5m | 1h | 1d) trong khoảng from..to
router.get('/sensor-data/chart', requireAuth, asyncRoute(async (req, res) => {
  const points = Number.parseInt(req.query.points, 10)
  if (points > 0) return res.json(await realtimeChart(points, req.query.sensorCode))

  const { from, to } = req.query
  if (from && to && new Date(from) >= new Date(to)) {
    return res.status(400).json({ message: 'from phải nhỏ hơn to' })
  }
  res.json(await bucketChart(req.query))
}))

async function realtimeChart(points, sensorCode) {
  const limit = Math.min(points, MAX_REALTIME_POINTS)
  const [rows] = await pool.query(
    `SELECT * FROM (
       SELECT sd.temperature, sd.humidity, sd.light, sd.recorded_at
       ${SENSOR_DATA_FROM}
       ${sensorCode ? 'WHERE s.sensor_code = ?' : ''}
       ORDER BY sd.recorded_at DESC
       LIMIT ?
     ) latest ORDER BY recorded_at ASC`,
    sensorCode ? [sensorCode, limit] : [limit]
  )
  return {
    labels: rows.map((r) => r.recorded_at.toLocaleTimeString('vi-VN', { hour12: false, timeZone: 'Asia/Ho_Chi_Minh' })),
    series: toChartSeries(rows),
    interval: 'raw',
    timezone: 'Asia/Ho_Chi_Minh'
  }
}
```

Hình 4.9. API dữ liệu cảm biến mới nhất và biểu đồ (routes/sensors.js)

src/routes/devices.js

```
router.post('/devices/:deviceId/control', requireAuth, asyncRoute(async (req, res) => {
  const { action, mode } = req.body || {}
  if (!VALID_ACTIONS.includes(action)) {
    return res.status(400).json({ message: 'action phải là ON hoặc OFF' })
  }

  const [[device]] = await pool.query(
    'SELECT device_id, device_code, device_name, location, status, mode FROM devices WHERE device_id = ?',
    [req.params.deviceId]
  )
  if (!device) return res.status(404).json({ message: 'Thiết bị không tồn tại' })

  const actionCode = crypto.randomUUID()
  const requestedMode = mode === 'AUTO' ? 'AUTO' : 'MANUAL'
  const now = new Date()

  // Ghi lịch sử với trạng thái PENDING, publish lệnh xuống ESP8266 rồi chờ ACK (luồng UC04):
  // có ACK -> mqttClient cập nhật devices + action_history = SUCCESS;
  // hết CONTROL_ACK_TIMEOUT_MS không có ACK -> action_history = FAILED, trạng thái đèn giữ nguyên.
  await pool.query(
    `INSERT INTO action_history (action_code, device_id, user_id, action, requested_state, result, area, action_time)
     VALUES (?, ?, ?, ?, ?, 'PENDING', ?, ?)`,
    [actionCode, device.device_id, req.user.userId, action, action, device.location, now]
  )

  // Chỉ bọc bước gửi lệnh / chờ ACK: lỗi khác (VD lỗi CSDL) đi tới middleware lỗi chung,
  // không bị báo nhầm là lỗi MQTT Broker
  let ack
  try {
    ack = await publishCommand({ actionCode, deviceCode: device.device_code, action, mode: requestedMode })
  } catch (err) {
    // Lỗi publish (mất kết nối Broker) thì timeout chưa kịp ghi FAILED, nên ghi ở đây
    await pool.query("UPDATE action_history SET result = 'FAILED' WHERE action_code = ?", [actionCode])
    return res.json({
      actionCode,
      deviceId: device.device_id,
      status: device.status,
      mode: device.mode,
      result: 'FAILED',
      message: FAILED_MESSAGES[err.message] || 'Không gửi được lệnh tới MQTT Broker'
    })
  }

  await pool.query('UPDATE devices SET mode = ? WHERE device_id = ?', [requestedMode, device.device_id])
  res.json({
    actionCode,
    deviceId: device.device_id,
    status: ack.status,
    mode: requestedMode,
    result: 'SUCCESS',
    confirmedAt: ack.confirmedAt
  })
}))
```

Hình 4.10. API điều khiển bật/tắt thiết bị (routes/devices.js)

src/routes/history.js

```
// Danh sách lịch sử điều khiển.
// Query: search, searchField, deviceId, action, result, from, to, page, size
router.get('/action-history', requireAuth, asyncRoute(async (req, res) => {
  const { page, size, offset } = parsePagination(req.query)
  const { deviceId, action, result, from, to, searchField = 'all' } = req.query
  const search = String(req.query.search || '').trim()

  const conditions = []
  const params = []
  const addFilter = (sql, value) => {
    conditions.push(sql)
    params.push(value)
  }

  if (search) {
    const s = searchCondition(search, searchField)
    conditions.push(s.sql)
    params.push(...s.params)
  }
  if (deviceId) addFilter('ah.device_id = ?', deviceId)
  if (action) addFilter('ah.action = ?', action)
  if (result) addFilter('ah.result = ?', result)
  if (from) addFilter('ah.action_time >= ?', new Date(from))
  if (to) addFilter('ah.action_time <= ?', new Date(to))
  const where = whereClause(conditions)

  const [[{ total }]] = await pool.query(
    `SELECT COUNT(*) AS total FROM action_history ah JOIN devices d ON d.device_id = ah.device_id ${where}`,
    params
  )
  const [rows] = await pool.query(
    `${HISTORY_COLUMNS} ${HISTORY_FROM}
     ${where}
     ORDER BY ah.action_time DESC, ah.history_id DESC
     LIMIT ? OFFSET ?`,
    [...params, size, offset]
  )

  res.json(buildPageResponse({ content: rows.map(toHistoryDto), page, size, totalElements: total }))
}))
```

Hình 4.11. API lịch sử điều khiển (routes/history.js)

src/mqttClient.js

```
export function connectMqtt() {
  client = mqtt.connect(process.env.MQTT_URL, {
    username: process.env.MQTT_USERNAME,
    password: process.env.MQTT_PASSWORD,
    clientId: `iot-thien-backend-${Math.random().toString(16).slice(2)}`,
    reconnectPeriod: 3000
  })

  client.on('connect', () => {
    console.log('[mqtt] connected to broker')
    client.subscribe(process.env.TOPIC_SENSOR_DATA, { qos: 1 })
    client.subscribe(process.env.TOPIC_DEVICE_STATUS, { qos: 1 })
    client.subscribe(process.env.TOPIC_HEARTBEAT, { qos: 0 })
  })

  client.on('reconnect', () => console.log('[mqtt] reconnecting...'))
  client.on('error', (err) => console.error('[mqtt] error:', err.message))

  client.on('message', async (topic, payloadBuf) => {
    const raw = payloadBuf.toString()
    try {
      if (matchTopic(topic, 'iot/sensors/+/data')) {
        await handleSensorData(raw)
      } else if (matchTopic(topic, 'iot/devices/+/status')) {
        await handleDeviceStatus(topic, raw)
      } else if (matchTopic(topic, 'iot/nodes/+/heartbeat')) {
        console.log(`[mqtt] heartbeat on ${topic}`)
      }
    } catch (err) {
      console.error(`[mqtt] failed to process message on ${topic}:`, err.message)
    }
  })

  return client
}
  ...
async function handleSensorData(raw) {
  const payload = JSON.parse(raw)
  const { sensorCode, temperature, humidity, light, timestamp } = payload

  if (
    typeof sensorCode !== 'string' ||
    typeof temperature !== 'number' ||
    typeof humidity !== 'number' ||
    typeof light !== 'number'
  ) {
    console.warn('[mqtt] invalid sensor payload, ignored:', raw)
    return
  }
  if (temperature < -40 || temperature > 125 || humidity < 0 || humidity > 100 || light < 0) {
    console.warn('[mqtt] sensor payload out of range, ignored:', raw)
    return
  }

  const [[sensor]] = await pool.query('SELECT sensor_id FROM sensors WHERE sensor_code = ?', [sensorCode])
  if (!sensor) {
    console.warn(`[mqtt] unknown sensorCode "${sensorCode}", ignored`)
    return
  }

  const status = classifyStatus({ temperature, humidity, light })
  const recordedAt = timestamp ? new Date(timestamp) : new Date()

  await pool.query(
    'INSERT INTO sensor_data (sensor_id, temperature, humidity, light, status, recorded_at) VALUES (?, ?, ?, ?, ?, ?)',
    [sensor.sensor_id, temperature, humidity, light, status, recordedAt]
  )
}

```

Hình 4.12. Kết nối MQTT và lưu dữ liệu cảm biến (mqttClient.js)

src/mqttClient.js

```
async function handleDeviceStatus(topic, raw) {
  const payload = JSON.parse(raw)
  const deviceCodeFromTopic = topic.split('/')[2]
  const { actionCode, deviceCode = deviceCodeFromTopic, status, result } = payload

  if (!['ON', 'OFF'].includes(status)) {
    console.warn('[mqtt] invalid device status payload, ignored:', raw)
    return
  }

  const [[device]] = await pool.query('SELECT device_id FROM devices WHERE device_code = ?', [deviceCode])
  if (!device) {
    console.warn(`[mqtt] unknown deviceCode "${deviceCode}", ignored`)
    return
  }

  const now = new Date()
  await pool.query('UPDATE devices SET status = ?, last_seen = ?, updated_at = ? WHERE device_id = ?', [
    status,
    now,
    now,
    device.device_id
  ])

  const pending = actionCode ? pendingActions.get(actionCode) : null
  if (pending && pending.deviceCode === deviceCode) {
    clearTimeout(pending.timer)
    pendingActions.delete(actionCode)
    // Mạch gửi result = REJECTED khi không thực hiện được lệnh -> route ghi FAILED
    if (result && result !== 'SUCCESS') {
      pending.reject(new Error('REJECTED'))
      return
    }
    await pool.query('UPDATE action_history SET result = ?, confirmed_at = ? WHERE action_code = ?', [
      'SUCCESS',
      now,
      actionCode
    ])
    pending.resolve({ status, confirmedAt: now.toISOString() })
  }
}

// Publish lệnh điều khiển tới ESP8266 và chờ ACK trong CONTROL_ACK_TIMEOUT_MS.
// Nếu hết thời gian chờ, action_history được cập nhật FAILED (đúng luồng UC04).
```

Hình 4.13. Xử lý phản hồi trạng thái (ACK) từ ESP8266 (mqttClient.js)

src/mqttClient.js

```
// Publish lệnh điều khiển tới ESP8266 và chờ ACK trong CONTROL_ACK_TIMEOUT_MS.
// Nếu hết thời gian chờ, action_history được cập nhật FAILED (đúng luồng UC04).
export function publishCommand({ actionCode, deviceCode, action, mode }) {
  const topic = `${process.env.TOPIC_DEVICE_COMMAND_PREFIX}/${deviceCode}/command`
  const payload = JSON.stringify({
    actionCode,
    deviceCode,
    action,
    mode,
    requestedAt: new Date().toISOString()
  })

  // Đang mất kết nối Broker thì báo lỗi ngay; nếu không, thư viện mqtt sẽ xếp hàng lệnh
  // và gửi muộn khi kết nối lại (đèn tự đổi trạng thái dù người dùng đã thấy "Thất bại")
  if (!client || !client.connected) {
    return Promise.reject(new Error('MQTT_DISCONNECTED'))
  }

  return new Promise((resolve, reject) => {
    const timeoutMs = Number(process.env.CONTROL_ACK_TIMEOUT_MS || 5000)
    const timer = setTimeout(async () => {
      pendingActions.delete(actionCode)
      try {
        await pool.query('UPDATE action_history SET result = ? WHERE action_code = ?', ['FAILED', actionCode])
      } catch (err) {
        console.error('[mqtt] failed to mark FAILED after timeout:', err.message)
      }
      reject(new Error('ACK_TIMEOUT'))
    }, timeoutMs)

    pendingActions.set(actionCode, { resolve, reject, timer, deviceCode })

    client.publish(topic, payload, { qos: 1 }, (err) => {
      if (err) {
        clearTimeout(timer)
        pendingActions.delete(actionCode)
        reject(err)
      }
    })
  })
}
```

Hình 4.14. Gửi lệnh điều khiển và chờ ACK (mqttClient.js)

src/utils/pagination.js

```
export function parsePagination(query) {
  const page = Math.max(0, Number.parseInt(query.page, 10) || 0)
  const size = Math.min(100, Math.max(1, Number.parseInt(query.size, 10) || 10))
  return { page, size, offset: page * size }
}

export function buildPageResponse({ content, page, size, totalElements }) {
  return {
    content,
    page,
    size,
    totalElements,
    totalPages: Math.max(1, Math.ceil(totalElements / size))
  }
}

// Không ghép trực tiếp tên cột do client gửi vào SQL - chỉ nhận giá trị nằm
// trong whitelist (đúng quy tắc 2.6.7 của báo cáo).
export function resolveSort(sortParam, whitelist, fallback) {
  const [field, dir] = String(sortParam || fallback).split(',')
  const safeField = whitelist.includes(field) ? field : fallback.split(',')[0]
  const safeDir = dir === 'asc' ? 'ASC' : 'DESC'
  return { field: safeField, dir: safeDir }
}
```

src/utils/sql.js

```
// Ghép danh sách điều kiện thành mệnh đề WHERE (rỗng nếu không có điều kiện nào)
export function whereClause(conditions) {
  return conditions.length ? `WHERE ${conditions.join(' AND ')}` : ''
}
```

Hình 4.15. Phân trang và ghép điều kiện SQL (utils/pagination.js, utils/sql.js)

src/utils/time.js

```
// Biểu thức SQL đổi cột thời gian sang giờ Việt Nam (UTC+7).
// Chỉ truyền vào tên cột cố định trong code, không truyền giá trị từ client.
function vnTime(column) {
  return `CONVERT_TZ(${column}, @@session.time_zone, '+07:00')`
}

// Điều kiện tìm CHÍNH XÁC theo thời gian (giờ 24h, giờ Việt Nam), dùng chung cho
// trang Cảm biến và Lịch sử hoạt động:
//   "17:30" / "17:30:15"                  -> đúng giờ:phút[:giây]
//   "17h"                                  -> cả khung giờ 17
//   "2026-10-07" (hoặc "07/10/2026")       -> cả ngày đó
//   "2026-10-07 17:30[:15]"                -> đúng ngày + giờ
// Dạng khác (VD "1", "abc") -> không khớp dòng nào.
// Trả về { sql, params } để ghép vào mệnh đề WHERE.
export function timeSearchCondition(column, search) {
  const time = vnTime(column)
  const clock = (h, m, s) => ({
    sql: [`HOUR(${time}) = ?`, `MINUTE(${time}) = ?`, ...(s === undefined ? [] : [`SECOND(${time}) = ?`])],
    params: [Number(h), Number(m), ...(s === undefined ? [] : [Number(s)])]
  })
  const join = (parts) => ({ sql: `(${parts.sql.join(' AND ')})`, params: parts.params })

  // Ngày dạng yyyy-MM-dd (định dạng hiển thị trên giao diện) hoặc dd/MM/yyyy
  const dateDMY = search.match(/^(\d{1,2})\/(\d{1,2})\/(\d{4})(?:\s+(\d{1,2}):(\d{1,2})(?::(\d{1,2}))?)?$/)
  const dateYMD = search.match(/^(\d{4})-(\d{1,2})-(\d{1,2})(?:\s+(\d{1,2}):(\d{1,2})(?::(\d{1,2}))?)?$/)

  if (dateDMY || dateYMD) {
    let year, month, day, h, m, s
    if (dateDMY) {
      [, day, month, year, h, m, s] = dateDMY
    } else {
      [, year, month, day, h, m, s] = dateYMD
    }
    const sql = [`DATE(${time}) = ?`]
    const params = [`${year}-${month.padStart(2, '0')}-${day.padStart(2, '0')}`]
    if (h !== undefined) {
      const c = clock(h, m, s)
      sql.push(...c.sql)
      params.push(...c.params)
    }
    return join({ sql, params })
  }

  const hm = search.match(/^(\d{1,2}):(\d{1,2})(?::(\d{1,2}))?$/)
  if (hm) return join(clock(hm[1], hm[2], hm[3]))

  const hourOnly = search.match(/^(\d{1,2})h$/i)
  if (hourOnly) return { sql: `(HOUR(${time}) = ?)`, params: [Number(hourOnly[1])] }

  return { sql: '(1 = 0)', params: [] }
}
```

Hình 4.16. Tìm kiếm chính xác theo thời gian (utils/time.js)

4.2.2. Thiết kế cơ sở dữ liệu:

Cơ sở dữ liệu MySQL gồm 5 bảng: users, sensors, devices, sensor_data và action_history. Bảng sensor_data liên kết với sensors; bảng action_history liên kết với devices và users.



Hình 4.17. Biểu đồ ERD

Ghi chú: ERD được tạo bằng chức năng Reverse Database của Visual Paradigm từ CSDL MySQL đang chạy; các cột kiểu ENUM (role, status, mode, result) được Visual Paradigm hiển thị dưới dạng varchar(n). Kiểu dữ liệu chính xác của từng cột được mô tả trong các bảng dưới đây.

Các bảng trong cơ sở dữ liệu:

- **users:** tài khoản người dùng.

Cột	Kiểu dữ liệu	Mô tả
user_id	INT (PK)	Mã người dùng, tự tăng
username	VARCHAR(50)	Tên đăng nhập, duy nhất
password_hash	VARCHAR(255)	Mật khẩu đã băm bằng bcrypt, không lưu mật khẩu gốc
full_name, student_id, email	VARCHAR	Họ tên, mã sinh viên, email
role	ENUM('USER','ADMIN')	Vai trò
github_link, figma_link, postman_link, pdf_link	VARCHAR(255)	Các đường dẫn hiển thị ở trang Profile
created_at, updated_at	TIMESTAMP	Thời điểm tạo, cập nhật

- **sensors:** danh sách cảm biến (mã SENSOR_01).

Cột	Kiểu dữ liệu	Mô tả
sensor_id	INT (PK)	Mã cảm biến, tự tăng
sensor_code	VARCHAR(30)	Mã cảm biến dùng trong topic MQTT, duy nhất
sensor_name, sensor_type, unit, location	VARCHAR	Tên, loại, đơn vị, vị trí
status	ENUM('ACTIVE','WARNING')	Trạng thái cảm biến
created_at	TIMESTAMP	Thời điểm tạo

- **devices:** danh sách thiết bị được điều khiển (mã LIGHT_01, tên "Đèn").

Cột	Kiểu dữ liệu	Mô tả
device_id	INT (PK)	Mã thiết bị, tự tăng
device_code	VARCHAR(30)	Mã thiết bị dùng trong topic MQTT, duy nhất
device_name, device_type, location	VARCHAR	Tên, loại thiết bị, vị trí
status	ENUM('ON','OFF')	Trạng thái hiện tại
mode	ENUM('AUTO','MANUAL')	Chế độ điều khiển
last_seen, updated_at	TIMESTAMP	Lần phản hồi gần nhất, lần cập nhật

- **sensor_data:** lịch sử dữ liệu đo, mỗi dòng là một lần đo.

Cột	Kiểu dữ liệu	Mô tả
data_id	BIGINT (PK)	Mã bản ghi, tự tăng
sensor_id	INT (FK)	Tham chiếu sensors.sensor_id
temperature	DECIMAL(5,2)	Nhiệt độ (°C), ràng buộc -40 đến 125
humidity	DECIMAL(5,2)	Độ ẩm (%), ràng buộc 0 đến 100
light	INT	Ánh sáng, không âm
status	ENUM('NORMAL','WARNING')	Trạng thái giá trị đo
recorded_at	TIMESTAMP	Thời điểm đo, có chỉ mục để tìm kiếm nhanh

- **action_history:** lịch sử điều khiển thiết bị.

Cột	Kiểu dữ liệu	Mô tả
history_id	BIGINT (PK)	Mã bản ghi, tự tăng
action_code	VARCHAR(36)	Mã lệnh (UUID) dùng để đối chiếu phản hồi từ ESP8266
device_id	INT (FK)	Tham chiếu devices.device_id
user_id	INT (FK)	Tham chiếu users.user_id (người điều khiển)
action, requested_state	VARCHAR(20)	Hành động và trạng thái yêu cầu (ON/OFF)
result	ENUM('PENDING','SUCCESS','FAILED')	Kết quả lệnh
area	VARCHAR(100)	Khu vực thiết bị
action_time, confirmed_at	TIMESTAMP	Thời điểm gửi lệnh, thời điểm xác nhận

4.2.3. MQTT topic sử dụng:

Bảng 4.4. Các MQTT topic

Topic	Publisher	Subscriber	QoS	Mục đích
iot/sensors/{sensorCode}/data	ESP8266	Backend	0	Gửi nhiệt độ, độ ẩm, ánh sáng
iot/devices/{deviceCode}/command	Backend	ESP8266	1	Gửi lệnh ON/OFF kèm actionCode
iot/devices/{deviceCode}/status	ESP8266	Backend	0	Phản hồi trạng thái thực tế (ACK)
iot/nodes/{nodeId}/heartbeat	ESP8266	Backend	0	Báo thiết bị đang hoạt động

Ví dụ dữ liệu trao đổi qua MQTT (đúng định dạng trong mã nguồn ESP8266):

```
// iot/sensors/SENSOR_01/data  (ESP8266 -> Backend)
{"sensorCode":"SENSOR_01","temperature":28.5,"humidity":65.0,"light":350,"timestamp":
"2026-10-07T17:30:15+07:00"}
// iot/devices/LIGHT_01/command  (Backend -> ESP8266)
{"actionCode":"1f0c...","deviceCode":"LIGHT_01","action":"ON","mode":"MANUAL"}
// iot/devices/LIGHT_01/status  (ESP8266 -> Backend)
{"actionCode":"1f0c...","deviceCode":"LIGHT_01","status":"ON","result":"SUCCESS","con
firmedAt":"2026-10-07T17:30:16+07:00"}
```

4.3. Xây dựng API Docs:

API Docs được xây dựng và công khai bằng Postman (tại liên kết https://www.postman.com/thienpham13112005aa-5408488/iot-thienpm/collection/tjp577w/iot-thien-api?action=share&creator=58995019, có liên kết trực tiếp tại trang Profile). Địa chỉ gốc của API là http://localhost:3000/api. Trừ API đăng nhập, mọi API đều yêu cầu header Authorization: Bearer <accessToken>.

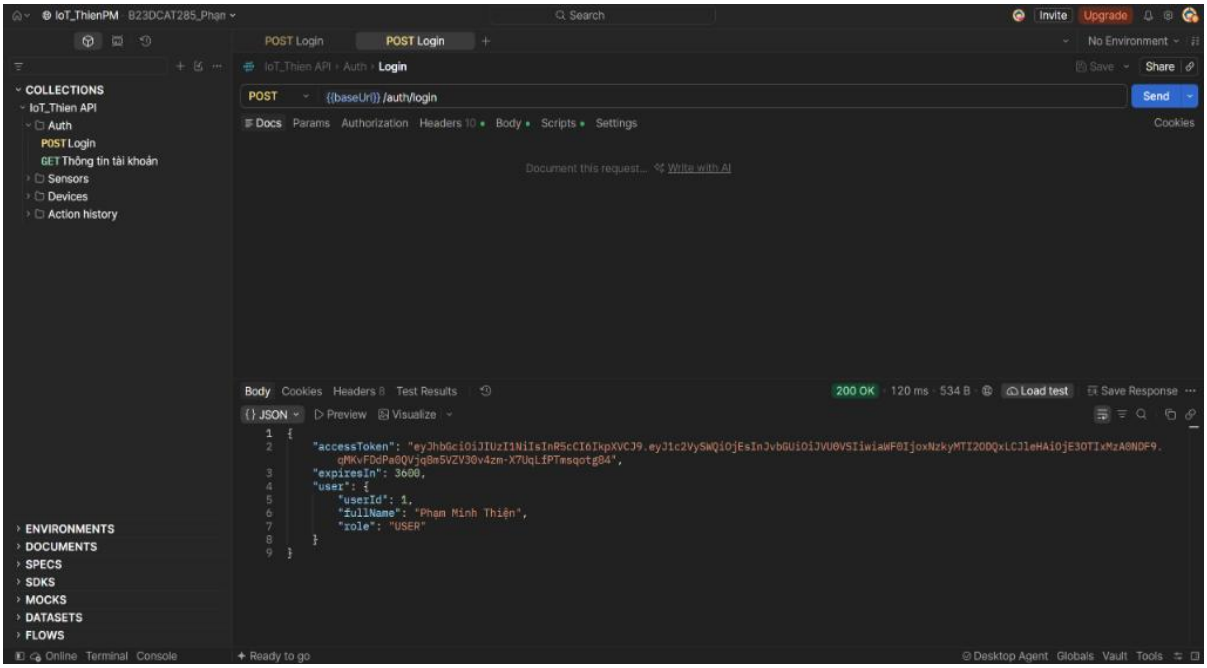
Bảng 4.5. Danh sách API

Method	Endpoint	Mô tả
POST	`/api/auth/login`	Đăng nhập, trả JWT
GET	`/api/auth/me`	Thông tin người dùng (trang Profile)
GET	`/api/sensor-data/latest`	Dữ liệu cảm biến mới nhất
GET	`/api/sensor-data/chart`	Dữ liệu biểu đồ
GET	`/api/sensor-data`	Lịch sử dữ liệu cảm biến (tìm kiếm, sắp xếp, phân trang)
GET	`/api/devices`	Danh sách thiết bị
POST	`/api/devices/{deviceId}/control`	Bật/tắt thiết bị
GET	`/api/action-history`	Lịch sử điều khiển (lọc, tìm kiếm, phân trang)

4.3.1. Đăng nhập:

- **Input:**
  - **Method:** POST
  - **Endpoint:** /api/auth/login
  - **Mô tả:** Kiểm tra username/password (bcrypt), trả JWT có thời hạn 1 giờ và thông tin người dùng.
  - **Tham số:**
    - Body JSON: {"username": "phamminhthien", "password": "123456"}
  - **Ví dụ:** http://localhost:3000/api/auth/login
- **Output:**

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "expiresIn": 3600,
  "user": { "userId": 1, "fullName": "Phạm Minh Thiện", "role": "USER" }
}
```

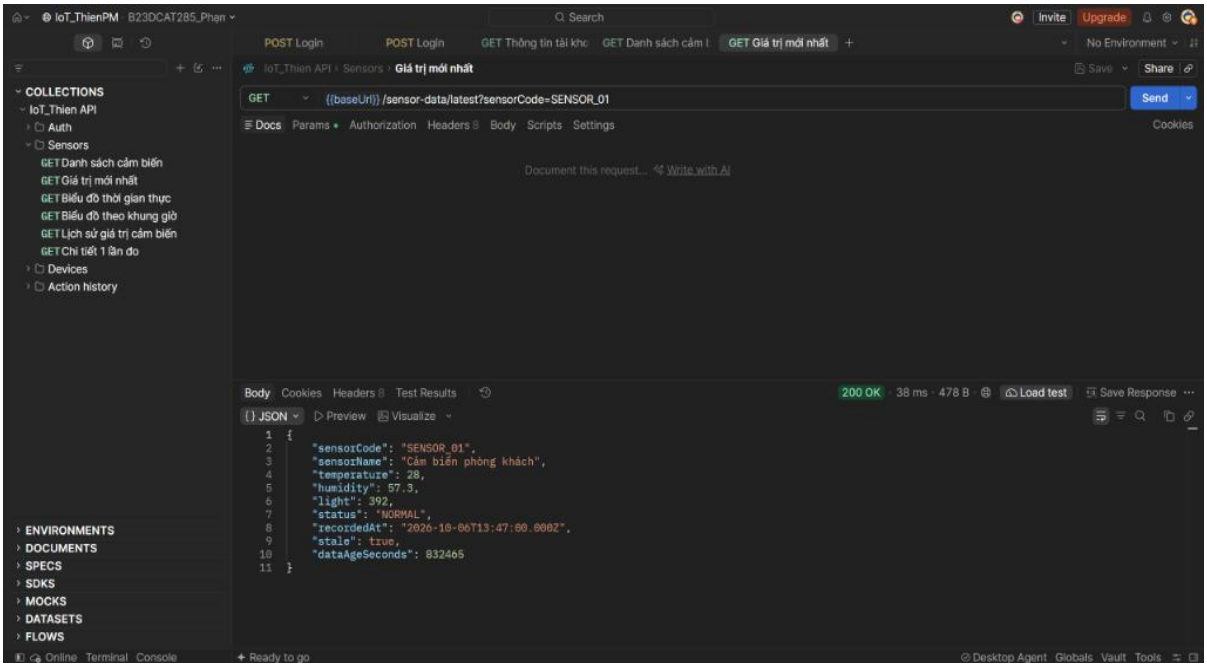


Hình 4.18. Kiểm thử API Đăng nhập trên Postman

4.3.2. Lấy dữ liệu cảm biến mới nhất:

- **Input:**
  - **Method:** GET
  - **Endpoint:** /api/sensor-data/latest
  - **Mô tả:** Trả về lần đo mới nhất của cảm biến, kèm độ trễ dữ liệu.
  - **Tham số:**
    - sensorCode = SENSOR_01
  - **Ví dụ:** http://localhost:3000/api/sensor-data/latest?sensorCode=SENSOR_01
- **Output:**

```
{
  "sensorCode": "SENSOR_01", "sensorName": "Cảm biến phòng khách",
  "temperature": 28, "humidity": 57.3, "light": 392,
  "status": "NORMAL", "recordedAt": "2026-10-06T13:47:00.000Z",
  "stale": true, "dataAgeSeconds": 1260610
}
```

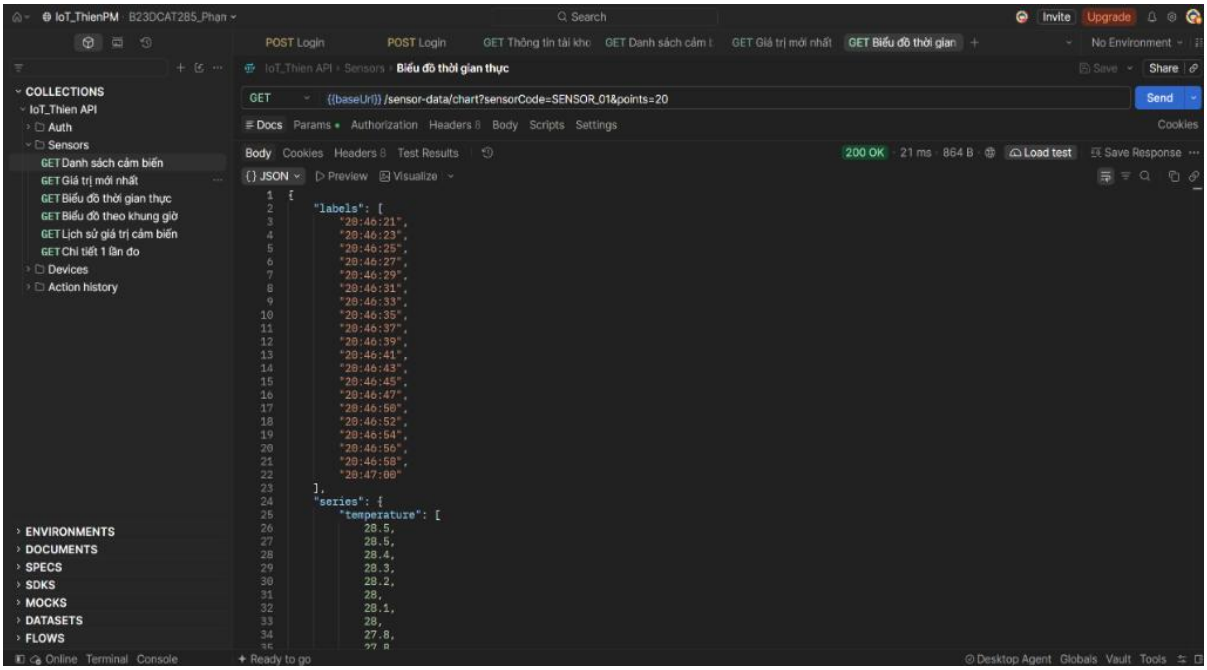


Hình 4.19. Kiểm thử API Lấy dữ liệu cảm biến mới nhất trên Postman

4.3.3. Lấy dữ liệu biểu đồ:

- **Input:**
  - **Method:** GET
  - **Endpoint:** /api/sensor-data/chart
  - **Mô tả:** Trả về N lần đo gần nhất (points) hoặc giá trị trung bình theo khoảng thời gian (from, to, interval = 1m/5m/1h/1d).
  - **Tham số:**
    - sensorCode = SENSOR_01
    - points = 20
  - **Ví dụ:** http://localhost:3000/api/sensor-data/chart?sensorCode=SENSOR_01&points=20
- **Output:**

```
{
  "labels": ["20:46:21", "20:46:23", "20:46:25", ..., "20:47:00"],
  "series": {
    "temperature": [28.5, 28.5, 28.4, ..., 28],
    "humidity": [58.3, 58.2, 58.1, ..., 57.3],
    "light": [392, 401, 406, ..., 392]
  },
  "interval": "raw", "timezone": "Asia/Ho_Chi_Minh"
}
```

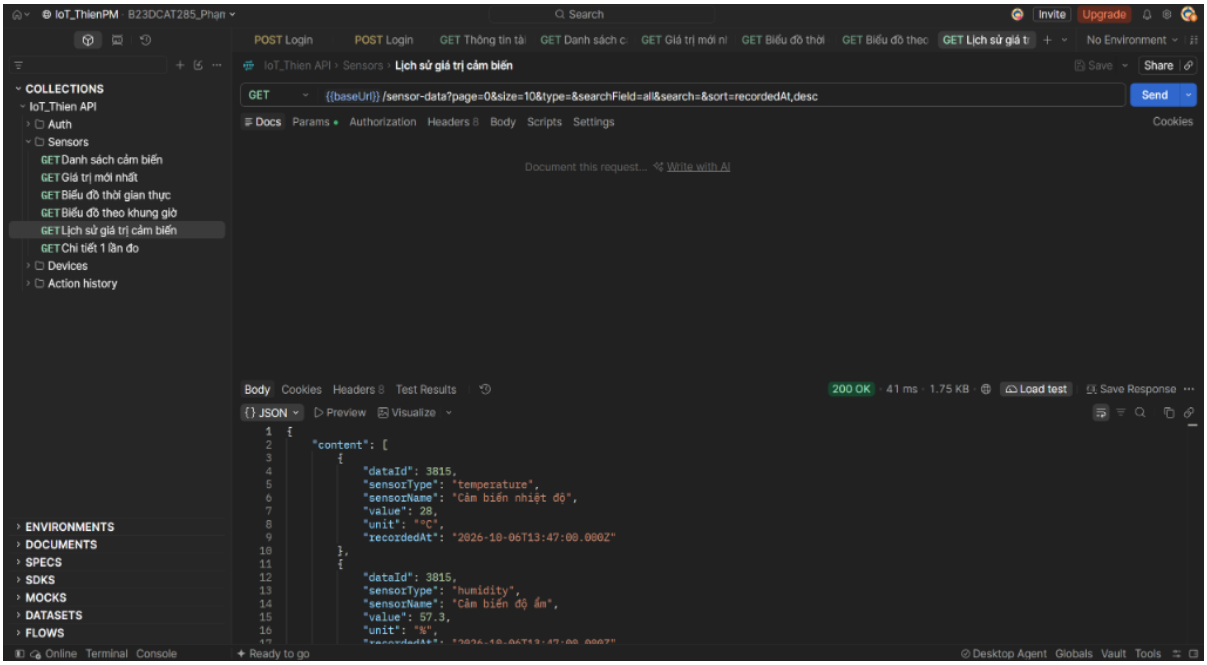


Hình 4.20. Kiểm thử API Lấy dữ liệu biểu đồ trên Postman

4.3.4. Lấy lịch sử dữ liệu cảm biến:

- **Input:**
  - **Method:** GET
  - **Endpoint:** /api/sensor-data
  - **Mô tả:** Trả về danh sách giá trị đo (mỗi dòng một loại cảm biến), hỗ trợ lọc, tìm kiếm, sắp xếp và phân trang.
  - **Tham số:**
    - type = temperature | humidity | light (bỏ trống: tất cả)
    - search = 350, searchField = value | time | id | all
    - sort = recordedAt,desc (sắp xếp theo recordedAt, value hoặc dataId)
    - page = 0, size = 10
  - **Ví dụ:** http://localhost:3000/api/sensor-data?type=light&search=350&searchField=value&page=0&size=10
- **Output:**

```
{
  "content": [
    { "dataId": 3764, "sensorType": "light", "sensorName": "Cảm biến ánh sáng",
      "value": 350, "unit": "lux", "recordedAt": "2026-10-06T13:45:17.000Z" }
  ],
  "page": 0, "size": 10, "totalElements": 1, "totalPages": 1
}
```

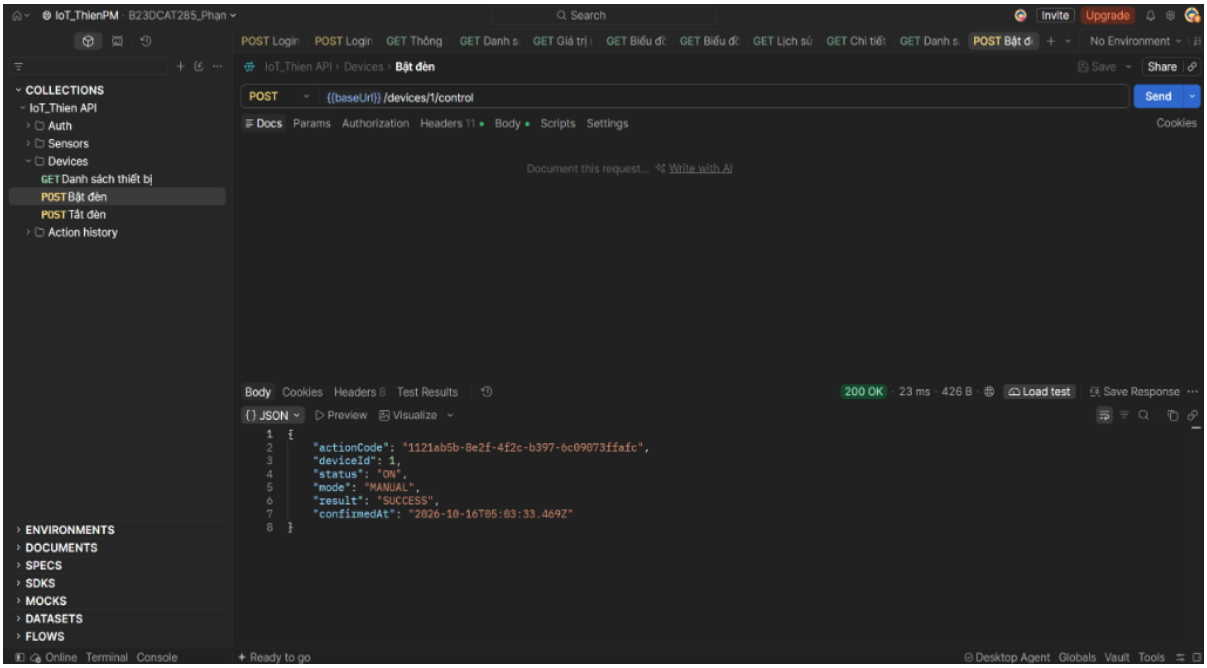


Hình 4.21. Kiểm thử API Lấy lịch sử dữ liệu cảm biến trên Postman

4.3.5. Điều khiển bật/tắt thiết bị:

- **Input:**
  - **Method:** POST
  - **Endpoint:** /api/devices/{deviceId}/control
  - **Mô tả:** Ghi lịch sử điều khiển (PENDING), publish lệnh xuống ESP8266 qua MQTT và chờ phản hồi: có ACK thì cập nhật trạng thái đèn và trả SUCCESS; quá 5 giây không có ACK thì trả FAILED, trạng thái đèn giữ nguyên.
  - **Tham số:**
    - deviceId = 1 (trên đường dẫn)
    - Body JSON: {"action": "ON", "mode": "MANUAL"}
  - **Ví dụ:** http://localhost:3000/api/devices/1/control
- **Output:**

```
{
  "actionCode": "42bfec3d-2b5a-4364-b4da-352d2b744d40", "deviceId": 1,
  "status": "ON", "mode": "MANUAL",
  "result": "SUCCESS", "confirmedAt": "2026-10-18T02:31:44.000Z"
}
```



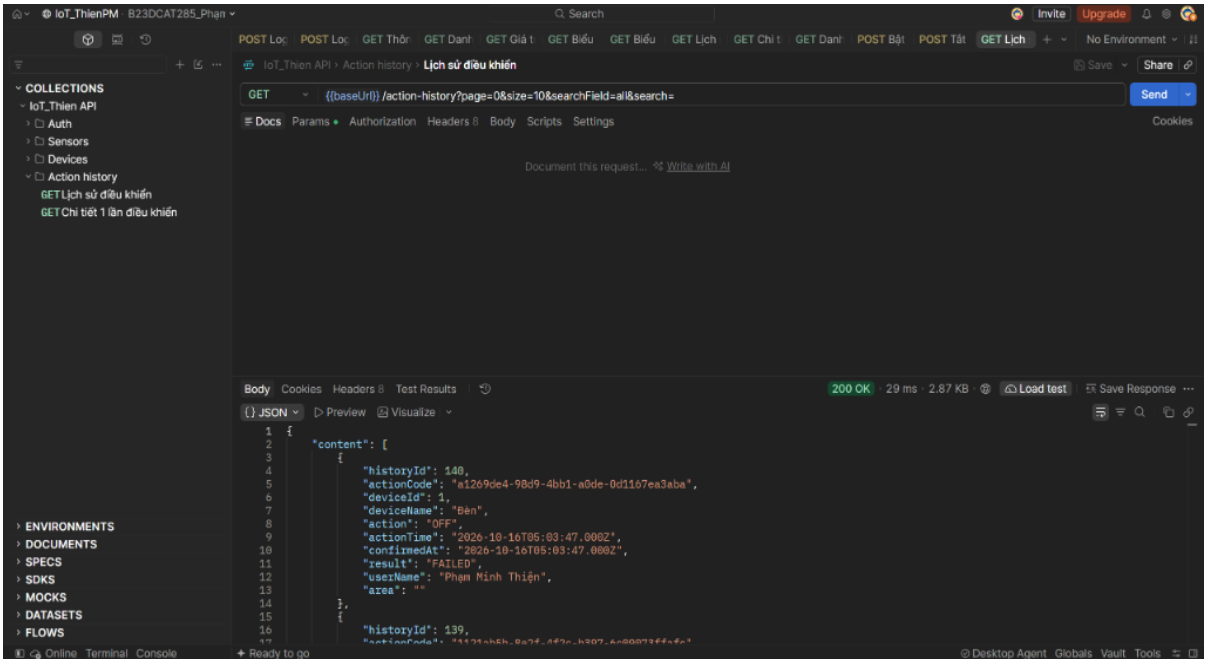
Hình 4.22. Kiểm thử API Điều khiển thiết bị trên Postman

4.3.6. Lấy lịch sử hoạt động:

- **Input:**
  - **Method:** GET
  - **Endpoint:** /api/action-history
  - **Mô tả:** Trả về lịch sử điều khiển, hỗ trợ lọc theo thiết bị, hành động, kết quả, tìm kiếm theo thời gian và phân trang.
  - **Tham số:**
    - deviceId = 1, action = ON | OFF, result = SUCCESS | FAILED | PENDING
    - search = 17:30, searchField = time
    - page = 0, size = 10
  - **Ví dụ:** http://localhost:3000/api/action-history?result=FAILED&page=0&size=10

- **Output:**

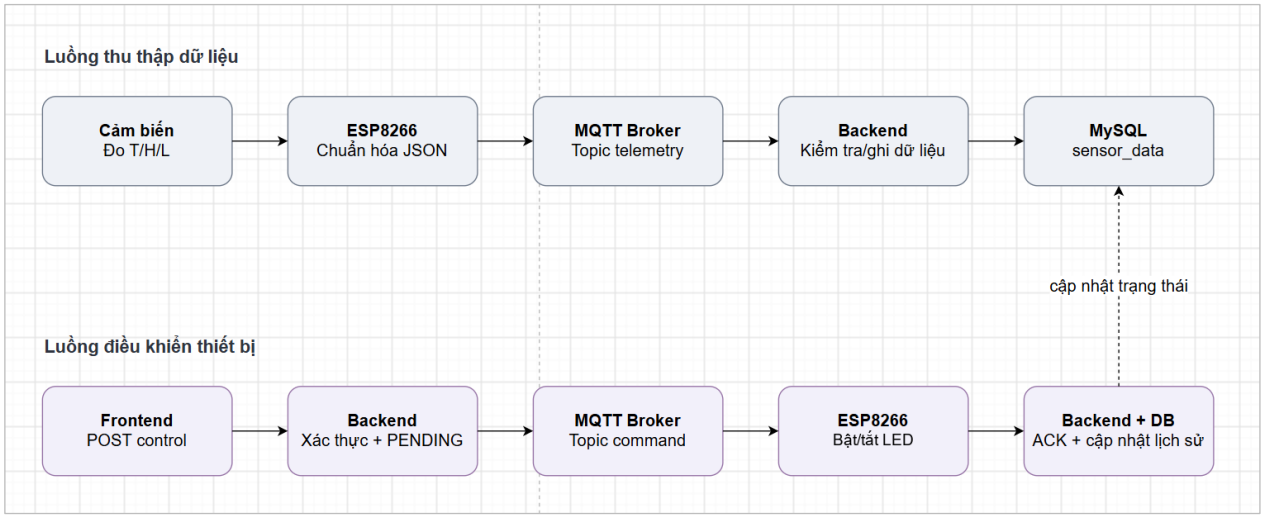

```
{
  "content": [
    { "historyId": 144, "actionCode": "c79b1b12-54d7-4a98-99b5-a81a5aa1aa7a",
      "deviceId": 1, "deviceName": "Đèn", "action": "OFF",
      "actionTime": "2026-10-16T07:59:31.000Z",
      "confirmedAt": "2026-10-16T07:59:31.000Z",
      "result": "FAILED", "userName": "Phạm Minh Thiện", "area": "" },
    ...
  ],
  "page": 0, "size": 10, "totalElements": 106, "totalPages": 11
}
```

Hình 4.23. Kiểm thử API Lấy lịch sử hoạt động trên Postman

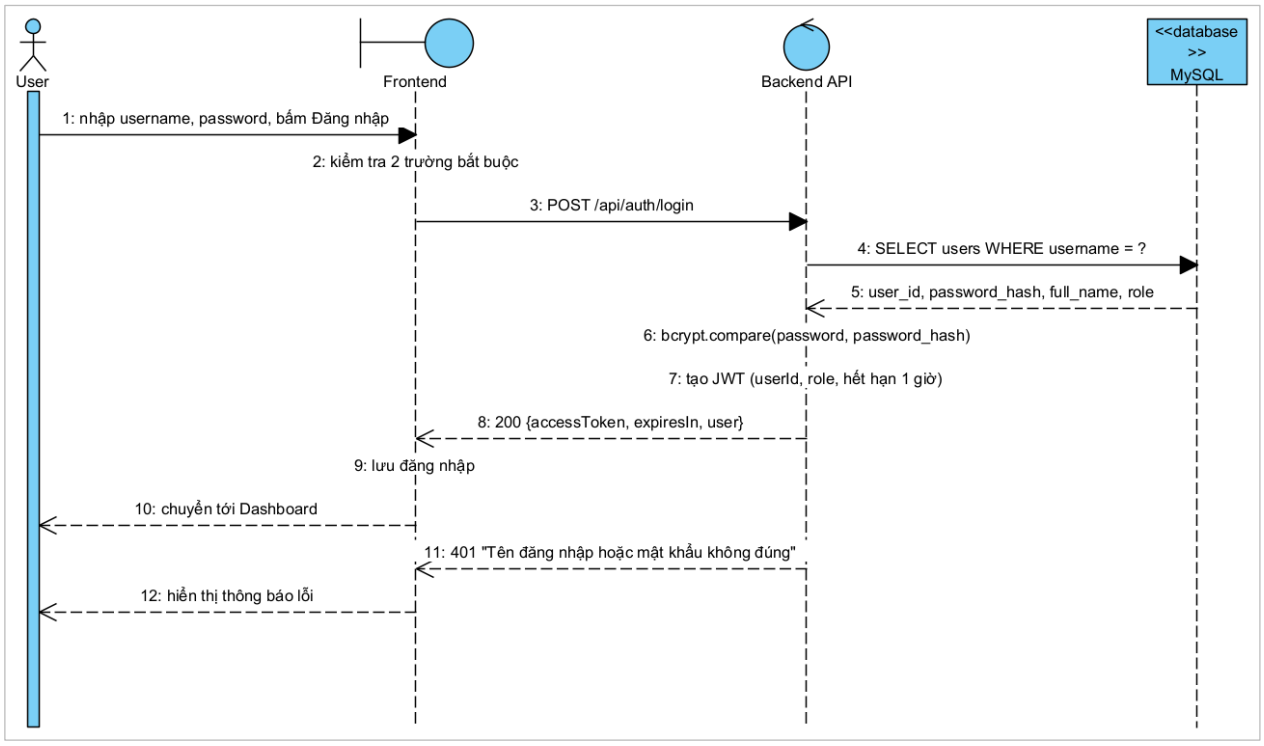
4.4. Biểu đồ tuần tự:

Hệ thống có hai luồng dữ liệu trọng tâm: luồng thu thập dữ liệu từ ESP8266 lên cơ sở dữ liệu và luồng điều khiển thiết bị từ giao diện xuống ESP8266.



Hình 4.24. Hai luồng dữ liệu trọng tâm của hệ thống

4.4.1. Đăng nhập:

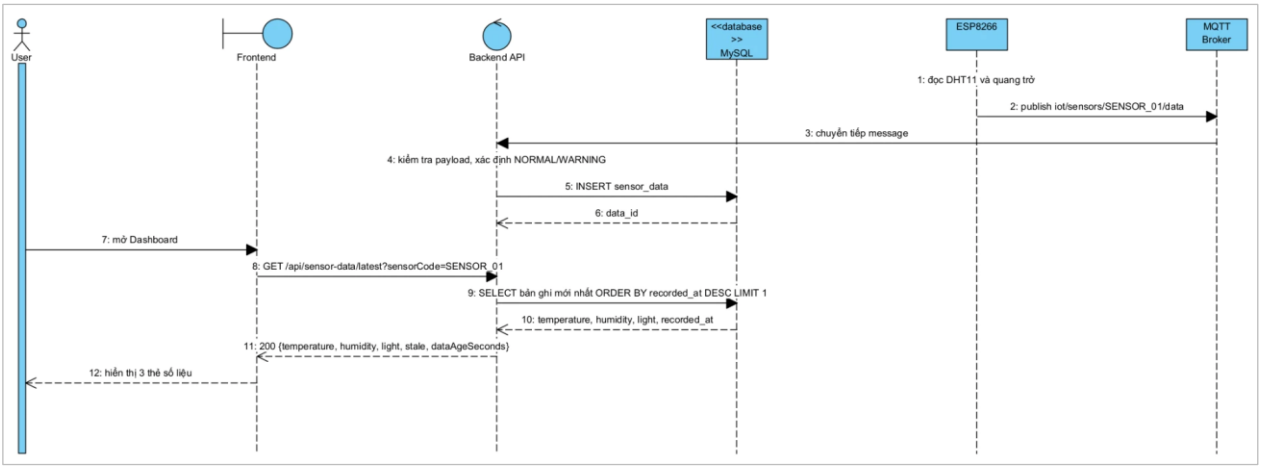


Hình 4.25. Biểu đồ tuần tự đăng nhập

- Bước 1–5: User nhập tài khoản; Frontend kiểm tra hai trường bắt buộc rồi gửi POST /api/auth/login; Backend truy vấn bảng users theo username.

- Bước 6: Backend so khớp mật khẩu bằng bcrypt.compare(). Sau đó chỉ xảy ra một trong hai trường hợp:
    - Đúng tài khoản (bước 7–10): Backend tạo JWT, trả HTTP 200; Frontend lưu đăng nhập và chuyển tới Dashboard.
    - Sai tài khoản hoặc mật khẩu (bước 11–12): Backend trả 401; Frontend hiển thị thông báo lỗi.

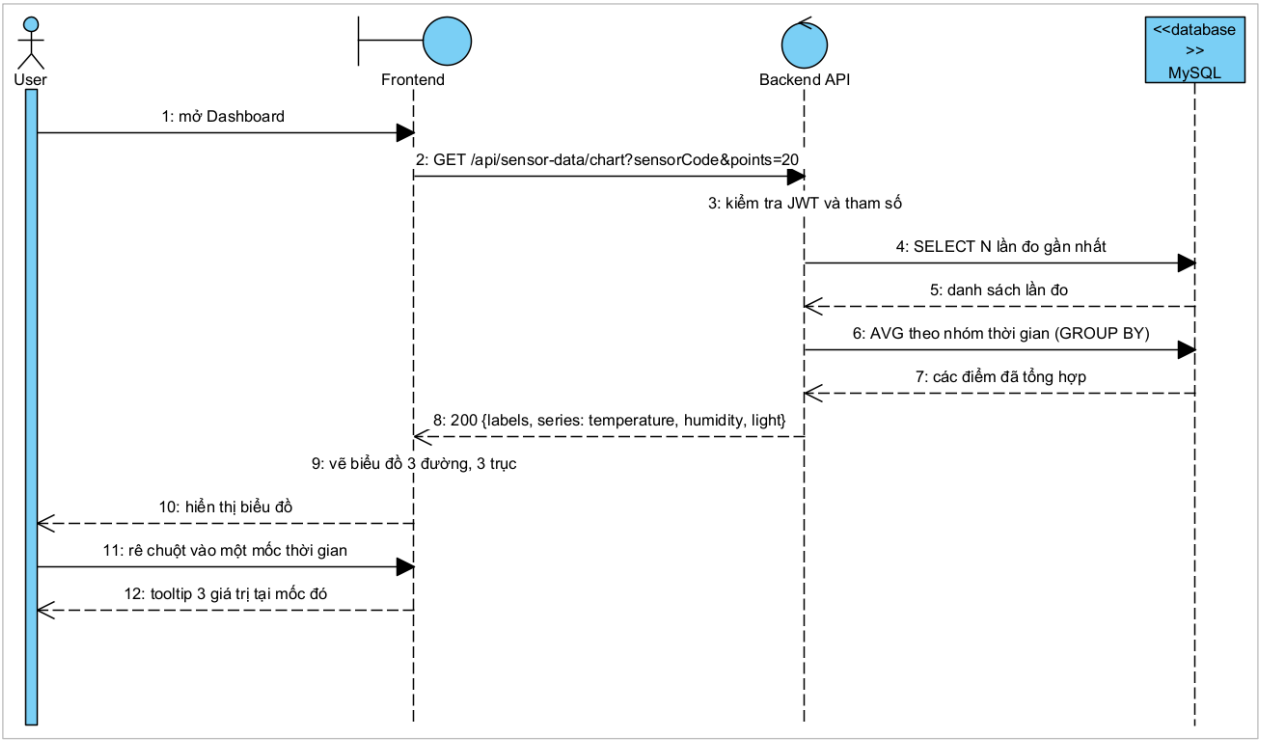
4.4.2. Xem dữ liệu cảm biến thời gian thực:



Hình 4.26. Biểu đồ tuần tự xem dữ liệu cảm biến thời gian thực

- Bước 1–6 lặp lại mỗi 2 giây: ESP8266 đọc cảm biến, publish lên topic iot/sensors/SENSOR_01/data; Broker chuyển tiếp tới Backend; Backend kiểm tra và lưu vào bảng sensor_data.
- Bước 7–12: User mở Dashboard; Frontend gọi GET /api/sensor-data/latest; Backend lấy bản ghi mới nhất và trả về để hiển thị lên 3 thẻ số liệu.

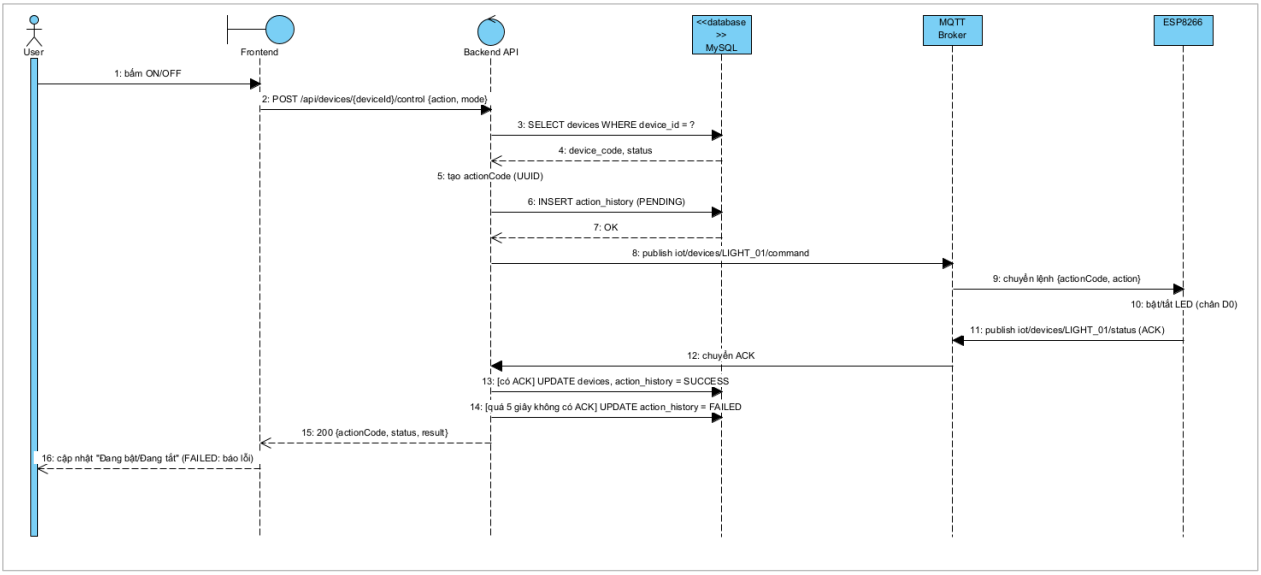
4.4.3. Xem biểu đồ dữ liệu cảm biến:



Hình 4.27. Biểu đồ tuần tự xem biểu đồ dữ liệu cảm biến

- Bước 1–3: Frontend gửi GET /api/sensor-data/chart; Backend kiểm tra JWT và tham số.
- Backend lấy dữ liệu theo một trong hai cách:
  - Có tham số points (bước 4–5): lấy N lần đo gần nhất.
  - Có from, to, interval (bước 6–7): tính trung bình theo nhóm thời gian bằng AVG và GROUP BY.
- Bước 8–12: Backend trả labels và ba chuỗi giá trị; Frontend vẽ biểu đồ, hiển thị tooltip khi User rê chuột.

4.4.4. Điều khiển bật/tắt thiết bị:

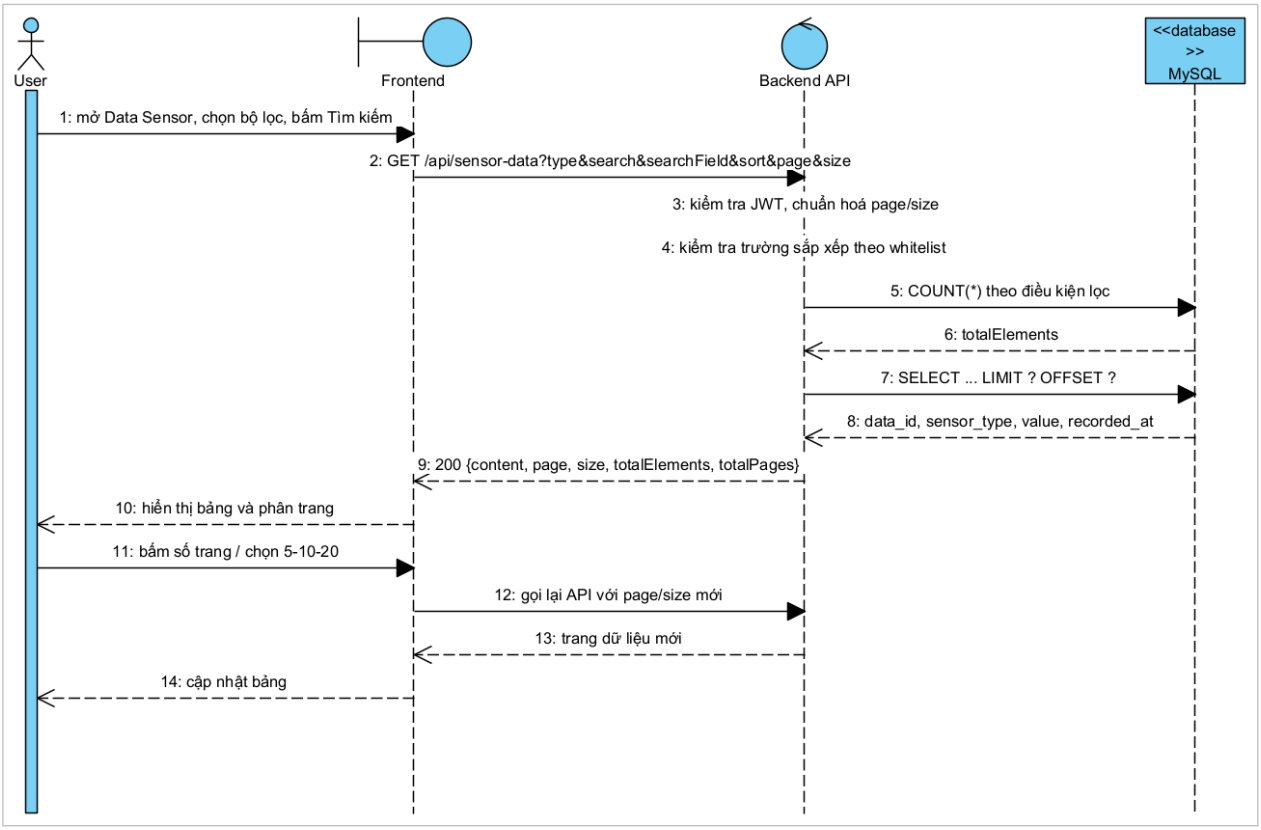


Hình 4.28. Biểu đồ tuần tự điều khiển bật/tắt thiết bị

- Bước 1–8: User bấm ON/OFF; Backend nhận yêu cầu điều khiển, tạo actionCode, ghi action_history với trạng thái PENDING rồi publish lệnh lên topic iot/devices/LIGHT_01/command.
- Bước 9–12: Broker chuyển lệnh tới ESP8266; ESP8266 bật/tắt LED ở chân D0 rồi publish phản hồi (ACK) lên topic status; Broker chuyển ACK về Backend.
- Sau đó chỉ xảy ra một trong hai trường hợp:
    - Có ACK (bước 13): cập nhật devices và action_history = SUCCESS.
    - Quá 5 giây không có ACK (bước 14): cập nhật action_history = FAILED, trạng thái đèn giữ nguyên.

- Bước 15–16: Backend trả kết quả (SUCCESS hoặc FAILED) cho Frontend; Frontend cập nhật trạng thái đèn, nếu thất bại thì hiển thị thông báo lỗi.

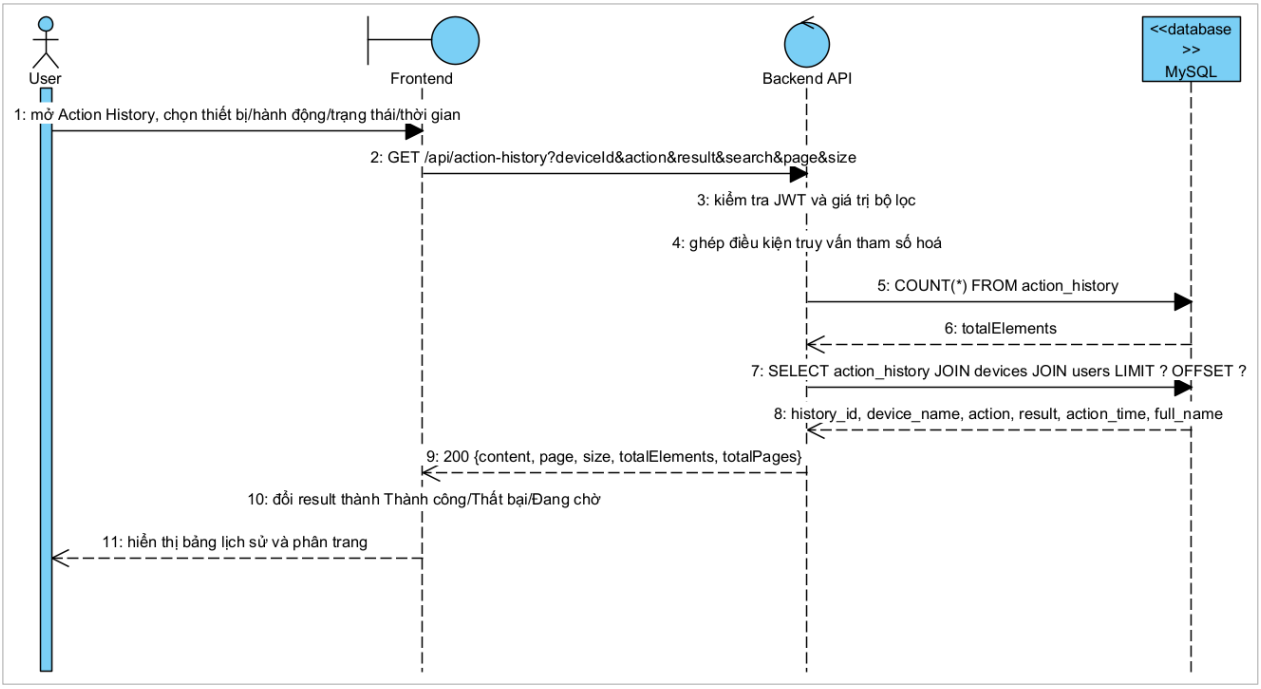
4.4.5. Xem lịch sử dữ liệu cảm biến:



Hình 4.29. Biểu đồ tuần tự xem lịch sử dữ liệu cảm biến

- Bước 1–4: Frontend gửi GET /api/sensor-data kèm tham số; Backend kiểm tra JWT, chuẩn hóa page/size và kiểm tra trường sắp xếp theo danh sách cho phép.
- Bước 5–10: Backend đếm tổng số bản ghi, lấy một trang bằng LIMIT/OFFSET và trả về; Frontend hiển thị bảng và phân trang.
- Bước 11–14 (khi User đổi trang hoặc số bản ghi mỗi trang): Frontend gọi lại API với page/size mới và cập nhật bảng.

4.4.6. Xem lịch sử hoạt động:



Hình 4.30. Biểu đồ tuần tự xem lịch sử hoạt động

- Bước 1–4: Frontend gửi GET /api/action-history kèm bộ lọc; Backend kiểm tra JWT và ghép điều kiện truy vấn tham số hóa.
- Bước 5–9: Backend đếm tổng số bản ghi, truy vấn action_history JOIN devices JOIN users và trả danh sách kèm thông tin phân trang.
- Bước 10–11: Frontend đổi kết quả sang Thành công/Thất bại/Đang chờ và hiển thị bảng lịch sử.

4.5. Front-end:

Trong xây dựng Front-end, hệ thống sử dụng ba công cụ nền tảng là HTML, CSS và JavaScript, viết thuần không dùng framework, chạy trực tiếp trên trình duyệt:

- **HTML** là bộ khung của năm trang (login.html, dashboard.html, sensors.html, history.html, profile.html); các vùng cần dữ liệu được đánh dấu bằng thuộc tính id.
- **CSS** (tệp css/style.css) quyết định màu sắc, bố cục, thẻ, bảng, badge trạng thái; bảng màu được khai báo bằng biến CSS để dùng thống nhất.
- **JavaScript** đảm nhận phần hành vi: đọc dữ liệu, điền vào các vùng theo id, vẽ biểu đồ bằng thư viện Chart.js, xử lý sự kiện bấm nút, lọc, tìm kiếm và phân trang.

Bảng 4.6. Tổ chức mã nguồn Front-end

Tệp	Chức năng
`js/config.js`	Địa chỉ API, số mốc biểu đồ, số bản ghi mỗi trang; hàm apiFetch gọi API kèm JWT
`js/data.js`	Các hàm gọi REST API: cảm biến, biểu đồ, thiết bị, lịch sử, hồ sơ
`js/auth.js`	Đăng nhập qua API, lưu JWT, đăng xuất, chặn truy cập khi chưa đăng nhập
`js/layout.js`	Phần chung: chữ cái avatar, đồng hồ trên thanh trên cùng
`js/table.js`	Bảng dữ liệu có phân trang dùng chung
`js/format.js`	Định dạng thời gian, chống chèn mã HTML
`js/pages/*.js`	Xử lý riêng của từng trang
`js/vendor/chart.umd.js`	Thư viện Chart.js 4.5.1

Mã nguồn các tệp JavaScript chính của Front-end:

js/config.js

```
// Hàm gọi API dùng chung kết nối với Backend
async function apiFetch(endpoint, options = {}) {
  const url = endpoint.startsWith('http')
    ? endpoint
    : `${API_BASE_URL}${endpoint.startsWith('/') ? '' : '/'}${endpoint}`
  const token = localStorage.getItem('iot_thien_token')

  const headers = {
    'Content-Type': 'application/json',
    ...(options.headers || {})
  }
  if (token) {
    headers.Authorization = `Bearer ${token}`
  }

  let response
  try {
    response = await fetch(url, {
      ...options,
      headers
    })
  } catch {
    // Backend chưa chạy / mất mạng: fetch báo "Failed to fetch", đổi sang thông báo tiếng Việt
    throw new Error('Không kết nối được máy chủ')
  }

  const data = await response.json().catch(() => ({}))

  // 401 khi đã có token: token hết hạn/không hợp lệ -> xoá phiên và về trang Đăng nhập.
  // 401 khi chưa có token (đang đăng nhập sai) thì giữ nguyên trang, hiện thông báo của Backend.
  if (response.status === 401 && token) {
    localStorage.removeItem('iot_thien_token')
    localStorage.removeItem('iot_thien_user')
    location.replace('login.html')
    throw new Error('Hết phiên đăng nhập, vui lòng đăng nhập lại')
  }

  if (!response.ok) {
    const errorMsg = data?.message || `Lỗi yêu cầu máy chủ (${response.status})`
    throw new Error(errorMsg)
  }

  return data
}
```

Hình 4.31. Hàm gọi API dùng chung apiFetch (js/config.js)

js/auth.js

```
// Đăng nhập / đăng xuất: lưu JWT token và thông tin người dùng trong localStorage

const TOKEN_KEY = 'iot_thien_token'
const USER_KEY = 'iot_thien_user'

// Đăng nhập qua REST API của Backend
async function login(username, password) {
  try {
    const res = await apiFetch('/auth/login', {
      method: 'POST',
      body: JSON.stringify({ username, password })
    })
    if (res.accessToken) {
      localStorage.setItem(TOKEN_KEY, res.accessToken)
      localStorage.setItem(USER_KEY, JSON.stringify(res.user))
      return { success: true, user: res.user }
    }
    return { success: false, message: 'Không nhận được mã xác thực token' }
  } catch (err) {
    return { success: false, message: err.message || 'Tên đăng nhập hoặc mật khẩu không đúng' }
  }
}

function logout() {
  localStorage.removeItem(TOKEN_KEY)
  localStorage.removeItem(USER_KEY)
}

function getToken() {
  return localStorage.getItem(TOKEN_KEY)
}

function getUser() {
  try {
    return JSON.parse(localStorage.getItem(USER_KEY) || 'null')
  } catch {
    return null
  }
}

function isLoggedIn() {
  return Boolean(getToken())
}

// Đặt ở đầu các trang cần đăng nhập: chưa đăng nhập thì chuyển về trang Đăng nhập
function requireLogin() {
  if (!isLoggedIn()) {
    location.replace('login.html')
  }
}
```

Hình 4.32. Đăng nhập, lưu JWT và chặn truy cập khi chưa đăng nhập (js/auth.js)

js/data.js

```
// Dịch vụ gọi API dữ liệu động từ Backend (Node.js/Express + MySQL)
// Toàn bộ dữ liệu tĩnh đã được loại bỏ, hệ thống truy vấn trực tiếp từ cơ sở dữ liệu.

// Lấy thông tin giá trị cảm biến mới nhất (Trang chủ)
async function getLatestSensor() {
  return await apiFetch('/sensor-data/latest')
}

// Lấy dữ liệu biểu đồ thời gian thực (Trang chủ - mặc định 20 mốc gần nhất)
async function getChartData(points = CHART_POINTS) {
  return await apiFetch(`/sensor-data/chart?points=${points}`)
}

// Lấy danh sách thiết bị (Trang chủ & Lọc lịch sử - hiện tại có 1 đèn LIGHT_01)
async function getDevices() {
  const res = await apiFetch('/devices')
  return res.devices || []
}

// Điều khiển bật / tắt thiết bị (Trang chủ)
async function setDeviceStatus(deviceId, action) {
  return await apiFetch(`/devices/${deviceId}/control`, {
    method: 'POST',
    body: JSON.stringify({ action, mode: 'MANUAL' })
  })
}

// Lấy danh sách dữ liệu cảm biến phân trang, tìm kiếm & lọc (Trang Cảm biến)
async function getSensorPage(params = {}) {
  const page = params.page ?? 0
  const size = params.size ?? 10
  const qs = new URLSearchParams()
  qs.set('page', page)
  qs.set('size', size)

  if (params.type === 'time') {
    if (params.search) {
      qs.set('search', params.search)
      qs.set('searchField', 'time')
    }
  } else {
    if (params.type) {
      qs.set('type', params.type)
    }
    if (params.search) {
      qs.set('search', params.search)
      qs.set('searchField', 'value')
    }
  }

  return await apiFetch(`/sensor-data?${qs.toString()}`)
}
```

Hình 4.33. Các hàm gọi REST API của Backend (js/data.js)

js/layout.js

```
// Phần dùng chung của các trang sau đăng nhập.
// Sidebar và topbar viết sẵn trong từng file .html; file này chỉ điền avatar + đồng hồ.

function initLayout() {
  requireLogin()

  const avatar = document.querySelector('.mini-avatar')
  if (avatar) avatar.textContent = initialOf(getUser()?.fullName)

  const clock = document.getElementById('clock')
  const tick = () => {
    if (clock) clock.textContent = formatClock(new Date())
  }
  tick()
  setInterval(tick, 30000)
}

// Hiện / ẩn khung lỗi đỏ (class form-error ở trang Đăng nhập)
function showError(element, message) {
  element.textContent = message || ''
  element.hidden = !message
}
```

js/format.js

```
// Hàm định dạng dùng chung

const pad = (n) => String(n).padStart(2, '0')

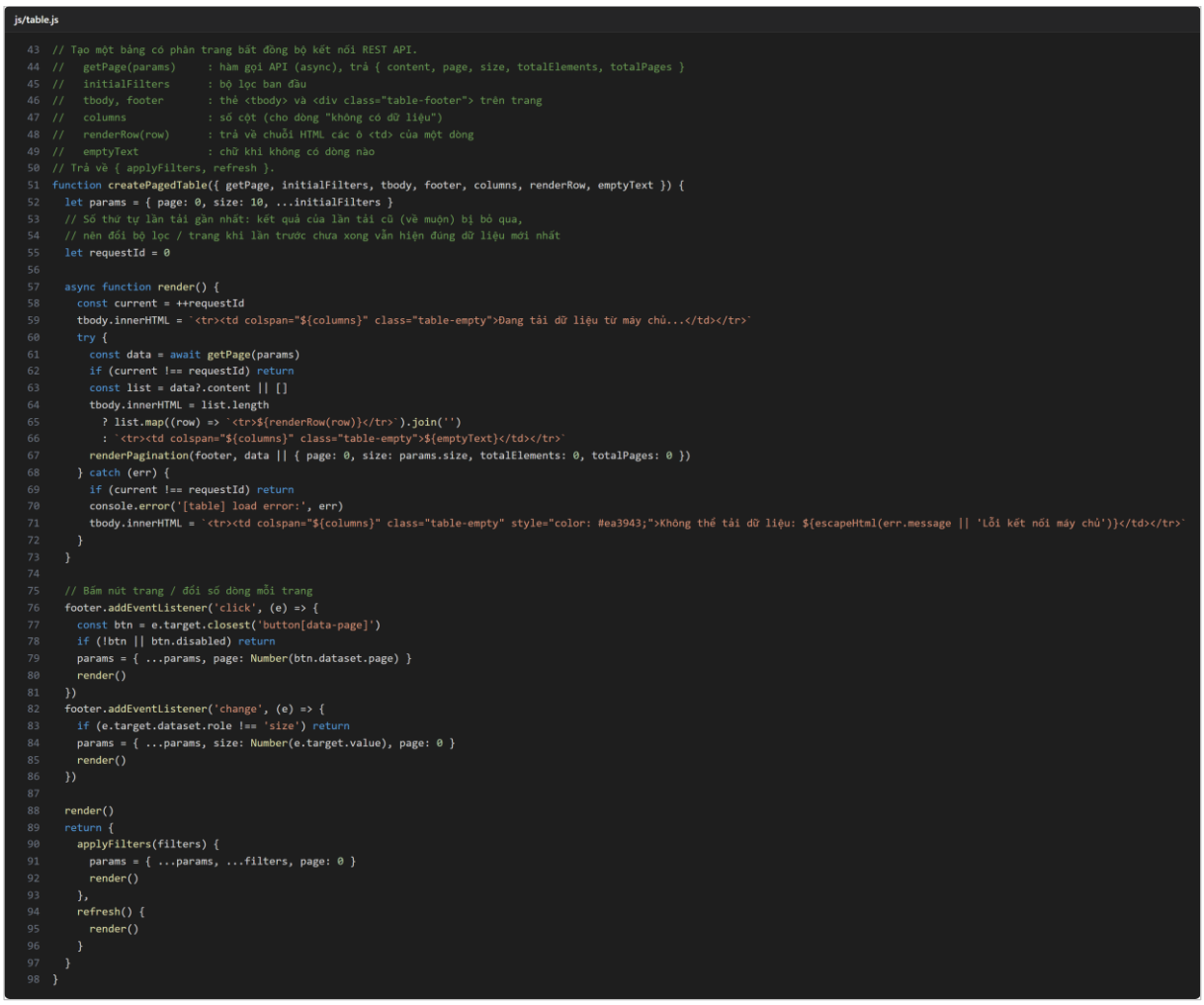
// yyyy-MM-dd HH:mm:ss (năm-tháng-ngày giờ:phút:giây) - trùng định dạng dùng khi tìm kiếm theo thời gian
function formatTime(value) {
  if (!value) return '--'
  const d = new Date(value)
  return `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())} ${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`
}

// HH:mm (24 giờ) - đồng hồ trên thanh trên cùng
function formatClock(date) {
  return `${pad(date.getHours())}:${pad(date.getMinutes())}`
}

// Chữ cái đầu của tên (từ cuối trong họ tên), dùng cho avatar: "Phạm Minh Thiện" -> "T"
function initialOf(fullName) {
  const name = String(fullName || '').trim().split(/\s+/).pop()
  return name ? name[0].toUpperCase() : '?'
}

// Chống chèn HTML: luôn dùng khi đưa dữ liệu vào innerHTML
function escapeHtml(value) {
  return String(value ?? '').replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' })[c])
}
```

Hình 4.34. Phần dùng chung và định dạng dữ liệu (js/layout.js, js/format.js)



```
js/table.js

// Tạo một bảng có phân trang bất đồng bộ kết nối REST API.
//   getPage(params)     : hàm gọi API (async), trả { content, page, size, totalElements, totalPages }
//   initialFilters      : bộ lọc ban đầu
//   tbody, footer       : thẻ <tbody> và <div class="table-footer"> trên trang
//   columns             : số cột (cho dòng "không có dữ liệu")
//   renderRow(row)      : trả về chuỗi HTML các ô <td> của một dòng
//   emptyText           : chữ khi không có dòng nào
// Trả về { applyFilters, refresh }.
function createPagedTable({ getPage, initialFilters, tbody, footer, columns, renderRow, emptyText }) {
  let params = { page: 0, size: 10, ...initialFilters }
  // Số thứ tự lần tải gần nhất: kết quả của lần tải cũ (về muộn) bị bỏ qua,
  // nên đổi bộ lọc / trang khi lần trước chưa xong vẫn hiện đúng dữ liệu mới nhất
  let requestId = 0

  async function render() {
    const current = ++requestId
    tbody.innerHTML = `<tr><td colspan="${columns}" class="table-empty">Đang tải dữ liệu từ máy chủ...</td></tr>`
    try {
      const data = await getPage(params)
      if (current !== requestId) return
      const list = data?.content || []
      tbody.innerHTML = list.length
        ? list.map((row) => `<tr>${renderRow(row)}</tr>`).join('')
        : `<tr><td colspan="${columns}" class="table-empty">${emptyText}</td></tr>`
      renderPagination(footer, data || { page: 0, size: params.size, totalElements: 0, totalPages: 0 })
    } catch (err) {
      if (current !== requestId) return
      console.error('[table] load error:', err)
      tbody.innerHTML = `<tr><td colspan="${columns}" class="table-empty" style="color: #ea3943;">Không thể tải dữ liệu: ${escapeHtml(err.message || 'Lỗi kết nối máy chủ')}</td></tr>`
    }
  }

  // Bấm nút trang / đổi số dòng mỗi trang
  footer.addEventListener('click', (e) => {
    const btn = e.target.closest('button[data-page]')
    if (!btn || btn.disabled) return
    params = { ...params, page: Number(btn.dataset.page) }
    render()
  })
  footer.addEventListener('change', (e) => {
    if (e.target.dataset.role !== 'size') return
    params = { ...params, size: Number(e.target.value), page: 0 }
    render()
  })

  render()
  return {
    applyFilters(filters) {
      params = { ...params, ...filters, page: 0 }
      render()
    },
    refresh() {
      render()
    }
  }
}
```

Hình 4.35. Bảng dữ liệu có phân trang dùng chung (js/table.js)

js/pages/login.js

```
// Trang Đăng nhập

// Đã đăng nhập rồi thì vào thẳng Trang chủ
if (isLoggedIn()) {
  location.replace('dashboard.html')
}

const usernameInput = document.getElementById('username')
const passwordInput = document.getElementById('password')
const toggleButton = document.getElementById('toggle-password')
const errorBox = document.getElementById('error')
const submitBtn = document.getElementById('submit')

// Điền sẵn tài khoản demo
usernameInput.value = DEMO_ACCOUNT.username
usernameInput.placeholder = DEMO_ACCOUNT.username
passwordInput.value = DEMO_ACCOUNT.password
document.getElementById('demo-hint').innerHTML =
  `Tài khoản demo: <b>${DEMO_ACCOUNT.username}</b> / <b>${DEMO_ACCOUNT.password}</b>`

// Nút Hiện / Ẩn mật khẩu
toggleButton.addEventListener('click', () => {
  const showing = passwordInput.type === 'text'
  passwordInput.type = showing ? 'password' : 'text'
  toggleButton.textContent = showing ? 'Hiện' : 'Ẩn'
})

// Gửi form đăng nhập qua API backend
document.getElementById('login-form').addEventListener('submit', async (e) => {
  e.preventDefault()
  showError(errorBox, '')
  submitBtn.disabled = true
  submitBtn.textContent = 'Đang đăng nhập...'

  try {
    const res = await login(usernameInput.value.trim(), passwordInput.value)
    if (res.success) {
      location.href = 'dashboard.html'
    } else {
      showError(errorBox, res.message || 'Tên đăng nhập hoặc mật khẩu không đúng')
    }
  } catch (err) {
    showError(errorBox, err.message || 'Lỗi kết nối máy chủ')
  } finally {
    submitBtn.disabled = false
    submitBtn.textContent = 'Đăng nhập'
  }
})
```

Hình 4.36. Xử lý trang Đăng nhập (js/pages/login.js)

js/pages/sensors.js

```
// Trang Cảm biến

initLayout()

// Tên hiển thị theo loại cảm biến
const SENSOR_NAMES = { temperature: 'Nhiệt độ', humidity: 'Độ ẩm', light: 'Ánh sáng' }

const typeSelect = document.getElementById('type')
const searchInput = document.getElementById('search')

// "Thời gian" không phải loại cảm biến: chọn nó để ô bên cạnh tìm theo thời gian (xem getSensorPage)
const table = createPagedTable({
  getPage: getSensorPage,
  initialFilters: { type: '', search: '' },
  tbody: document.getElementById('rows'),
  footer: document.getElementById('footer'),
  columns: 4,
  emptyText: 'Không có bản ghi phù hợp.',
  renderRow: (row) => `
    <td class="technical-id">${row.dataId}</td>
    <td>${SENSOR_NAMES[row.sensorType]}</td>
    <td>${row.value} ${row.unit}</td>
    <td>${formatTime(row.recordedAt)}</td>`
})

// Chọn "Thời gian" thì đổi nhãn + gợi ý của 2 ô ngay (chưa cần bấm Tìm kiếm)
function updateLabels() {
  const byTime = typeSelect.value === 'time'
  document.getElementById('type-label').textContent = byTime ? 'TÌM THEO' : 'LOẠI CẢM BIẾN'
  document.getElementById('search-label').textContent = byTime ? 'THỜI GIAN' : 'GIÁ TRỊ'
  searchInput.placeholder = byTime ? 'Nhập thời gian (VD: 2026-10-07 hoặc 10:30)...' : 'Nhập giá trị cần tìm...'
}
typeSelect.addEventListener('change', updateLabels)

// Tìm kiếm: lấy giá trị đang chọn/gõ và áp bộ lọc
document.getElementById('filter-form').addEventListener('submit', (e) => {
  e.preventDefault()
  table.applyFilters({ type: typeSelect.value, search: searchInput.value.trim() })
})

// Đặt lại: xoá ô lọc và bỏ bộ lọc
document.getElementById('reset').addEventListener('click', () => {
  typeSelect.value = ''
  searchInput.value = ''
  updateLabels()
  table.applyFilters({ type: '', search: '' })
})
```

Hình 4.37. Xử lý trang Data Sensor (js/pages/sensors.js)

js/pages/history.js

```
// Trang Lịch sử hoạt động - Dữ liệu động kết nối REST API

initLayout()

// Cột "Trạng thái": Thành công / Thất bại / Đang chờ + màu badge
const RESULTS = {
  SUCCESS: { label: 'Thành công', tone: 'good' },
  FAILED: { label: 'Thất bại', tone: 'bad' },
  PENDING: { label: 'Đang chờ', tone: 'warn' }
}

function resultBadge(result) {
  const { label, tone } = RESULTS[result] || { label: result, tone: 'neutral' }
  return `<span class="status-badge ${tone}"><i></i>${escapeHtml(label)}</span>`
}

const deviceSelect = document.getElementById('device')
const actionSelect = document.getElementById('action')
const resultSelect = document.getElementById('result')
const searchInput = document.getElementById('search')

const table = createPagedTable({
  getPage: getActionPage,
  initialFilters: { deviceId: '', action: '', result: '', search: '' },
  tbody: document.getElementById('rows'),
  footer: document.getElementById('footer'),
  columns: 6,
  emptyText: 'Không có dữ liệu phù hợp.',
  renderRow: (row) => `
    <td class="technical-id">${row.historyId}</td>
    <td><strong>${escapeHtml(row.deviceName || 'Đèn')}</strong></td>
    <td>${row.action}</td>
    <td>${resultBadge(row.result)}</td>
    <td>${formatTime(row.actionTime)}</td>
    <td>${escapeHtml(row.userName || 'Phạm Minh Thiện')}</td>`
})

// Nạp danh sách thiết bị động từ Backend vào ô lọc THIẾT BỊ
async function populateDevices() {
  try {
    const devices = await getDevices()
    for (const d of devices) {
      deviceSelect.add(new Option(d.deviceName || 'Đèn', d.deviceId))
    }
  } catch (err) {
    console.error('[history] populateDevices error:', err)
  }
}
populateDevices()

document.getElementById('filter-form').addEventListener('submit', (e) => {
  e.preventDefault()
  table.applyFilters({
    deviceId: deviceSelect.value,
    action: actionSelect.value,
    result: resultSelect.value,
    search: searchInput.value.trim()
  })
})

document.getElementById('reset').addEventListener('click', () => {
  deviceSelect.value = ''
  actionSelect.value = ''
  resultSelect.value = ''
  searchInput.value = ''
  table.applyFilters({ deviceId: '', action: '', result: '', search: '' })
})
```

Hình 4.38. Xử lý trang Action History (js/pages/history.js)

js/pages/profile.js

```
// Trang Hồ sơ - Dữ liệu động kết nối REST API

initLayout()

async function loadProfile() {
  try {
    const profile = await getProfile()
    if (!profile) return

    // Điền thông tin vào các thẻ có data-field="..."
    document.querySelectorAll('[data-field]').forEach((el) => {
      const field = el.dataset.field
      if (profile[field] !== undefined) {
        el.textContent = profile[field]
      }
    })

    const avatar = document.getElementById('avatar')
    if (avatar && profile.fullName) {
      avatar.textContent = initialOf(profile.fullName)
    }

    if (profile.githubLink) document.getElementById('link-github').href = profile.githubLink
    if (profile.figmaLink) document.getElementById('link-figma').href = profile.figmaLink
    if (profile.postmanLink) document.getElementById('link-postman').href = profile.postmanLink
    if (profile.pdfLink) document.getElementById('link-pdf').href = profile.pdfLink
  } catch (err) {
    console.error('[profile] load error:', err)
  }
}

loadProfile()

document.getElementById('logout').addEventListener('click', () => {
  logout()
  location.href = 'login.html'
})
```

Hình 4.39. Xử lý trang Profile (js/pages/profile.js)

Ví dụ mã nguồn hiển thị và điều khiển đèn trên Dashboard: hàm renderDevices dựng thẻ cho đèn, gán class màu cho nút ON/OFF theo trạng thái; khi bấm nút, chương trình gọi API POST /api/devices/{deviceId}/control rồi tải lại trạng thái đèn từ Backend và vẽ lại.

js/pages/dashboard.js

```
 97  function renderDevices() {
 98    const container = document.getElementById('device-list')
     ...
110    container.innerHTML = currentDevices
111      .map((device) => {
112        const isOn = device.status === 'ON'
113        return `
114          <div class="device-card">
115            <div class="device-copy">
116              <strong>${escapeHtml(device.deviceName || 'Đèn')}</strong>
117            </div>
118            <div class="device-status">${isOn ? 'Đang bật' : 'Đang tắt'}</div>
119            <div class="device-actions">
120              <button class="mode-button on ${isOn ? 'active' : ''}"
121                data-id="${device.deviceId}" data-action="ON" ${isControlling ? 'disabled' : ''}>ON</button>
122              <button class="mode-button off ${!isOn ? 'active' : ''}"
123                data-id="${device.deviceId}" data-action="OFF" ${isControlling ? 'disabled' : ''}>OFF</button>
124            </div>
125          </div>`
126      })
127      .join('')
     ...
130  // Bấm nút ON / OFF điều khiển đèn qua REST API
131  document.getElementById('device-list').addEventListener('click', async (e) => {
132    const btn = e.target.closest('button[data-action]')
133    if (!btn || isControlling) return
134    const deviceId = Number(btn.dataset.id)
135    const action = btn.dataset.action
136
137    isControlling = true
138    const target = currentDevices.find((d) => d.deviceId === deviceId)
139    if (target) {
140      target.status = action
141      renderDevices()
142    }
143
144    try {
145      const res = await setDeviceStatus(deviceId, action)
146      await loadDevices()
147      if (res.result !== 'SUCCESS') alert(`Điều khiển thất bại: ${res.message || 'Thiết bị không phản hồi'}`)
     ...
153      isControlling = false
154      renderDevices()
155    }
156  })
```

Hình 4.40. Mã nguồn hiển thị và điều khiển đèn (dashboard.js)

4.6. Lấy dữ liệu từ server:

Front-end tương tác với Backend bằng HTTP request thông qua hàm fetch của JavaScript. Mọi lời gọi đi qua hàm dùng chung apiFetch (tệp js/config.js): ghép địa chỉ gốc http://localhost:3000/api, đính kèm JWT trong header Authorization, đọc phản hồi JSON; nếu máy chủ trả 401 (token hết hạn hoặc không hợp lệ), Front-end xóa phiên đăng nhập và chuyển về trang Đăng nhập. Các hàm trong js/data.js gọi đúng các API ở mục 4.3. Trên Dashboard, thẻ số liệu và biểu đồ được làm mới mỗi 5 giây; bảng dữ liệu cảm biến và lịch sử hoạt động thực hiện phân trang, tìm kiếm và lọc trực tiếp trên cơ sở dữ liệu MySQL thông qua tham số truy vấn.

CHƯƠNG 5: KẾT LUẬN

Qua quá trình nghiên cứu và thực hiện, bài tập lớn môn học "IoT và Ứng dụng" đã xây dựng được hệ thống IoT_ThienPM gồm đầy đủ các thành phần: phần cứng ESP8266, MQTT Broker, Backend, cơ sở dữ liệu và giao diện Web.

5.1. Các kết quả đã đạt được:

- **Phần cứng:** mạch ESP8266 đọc được nhiệt độ, độ ẩm, ánh sáng, gửi dữ liệu theo chu kỳ 2 giây qua MQTT, nhận lệnh bật/tắt LED và gửi phản hồi trạng thái; tự kết nối lại khi mất Wi-Fi hoặc MQTT.
- **Backend và cơ sở dữ liệu:** thiết kế 5 bảng có ràng buộc, chỉ mục; xây dựng REST API có xác thực JWT, mật khẩu băm bằng bcrypt, hỗ trợ tìm kiếm, sắp xếp, phân trang; API Docs trên Postman.
- **Giao diện:** hoàn thiện năm trang Đăng nhập, Dashboard, Data Sensor, Action History, Profile; biểu đồ ba trục có tooltip; điều khiển đèn với phản hồi trạng thái bằng màu sắc; tìm kiếm chính xác theo giá trị và nhiều định dạng thời gian.

5.2. Hướng phát triển trong tương lai:

- **Nâng cấp giao tiếp thời gian thực:** chuyển đổi cơ chế định kỳ (polling) sang sử dụng WebSocket hoặc MQTT over WebSockets để nhận dữ liệu cảm biến đẩy về tức thì mà không cần gửi request liên tục.
- **Tự động hóa:** xây dựng kịch bản tự bật/tắt thiết bị theo ngưỡng cảm biến, cảnh báo khi giá trị vượt ngưỡng.
- **Mở rộng:** bổ sung thêm cảm biến, relay để điều khiển thiết bị thật; dùng cảm biến ánh sáng BH1750 để đo lux chính xác.
- **Triển khai:** đưa hệ thống lên máy chủ, bảo mật Broker bằng tài khoản riêng và TLS.

Bài tập lớn không chỉ là một sản phẩm ứng dụng mà còn giúp em củng cố kiến thức từ lý thuyết đến thực hành, bao gồm lập trình nhúng, giao thức MQTT, phát triển Backend, thiết kế cơ sở dữ liệu và xây dựng giao diện Web.

Github dự án: https://github.com/thienpham205/IoT

Figma: https://www.figma.com/design/FUkx7yZ5Nas3ofLx7wTZ9p/IOT_THIEN

Postman API Docs: https://www.postman.com/thienpham13112005aa-5408488/iot-thienpm/collection/tjp577w/iot-thien-api?action=share&creator=58995019